C++ Standard Library Issues to be moved in Oulu

Doc. no. P0165R2

Date: Revised 2016-05-30 at 02:05:30 UTC

Project: Programming Language C++
Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

2181. Exceptions from *seed sequence* operations

Section: 26.6.1.2 [rand.req.seedseq], 26.6.3 [rand.eng], 26.6.4 [rand.adapt] **Status:** Ready **Submitter:** Daniel Krügler **Opened:** 2012-08-18 **Last modified:** 2016-03-06

Priority: 3

View all other issues in [rand.req.seedseq].

View all issues with Ready status.

Discussion:

LWG issue 2180 points out some deficiences in regard to the specification of the library-provided type `std::seed_seq` regarding exceptions, but there is another specification problem in regard to general types satisfying the *seed sequence* constraints (named `SSeq`) as described in 26.6.1.2 [rand.req.seedseq].

26.6.3 [rand.eng] p3 and 26.6.4.1 [rand.adapt.general] p3 say upfront:

> Except where specified otherwise, no function described in this section 26.6.3 [rand.eng]/26.6.4 [rand.adapt] throws an exception.

This constraint causes problems, because the described templates in these sub-clauses depend on operations of `SSeq::generate()` which is a function template, that depends both on operations provided by the implementor of `SSeq` (e.g. of `std::seed_seq`), and those of the random access iterator type provided by the caller. With class template `linear_congruential_engine` we have just one example for a user of `SSeq::generate()` via:

```
template<class Sseq>
linear_congruential_engine<>::linear_congruential_engine(Sseq& q);

template<class Sseq>
void linear_congruential_engine<>::seed(Sseq& q);
```

None of these operations has an exclusion rule for exceptions.

As described in 2180 the wording for `std::seed_seq` should and can be fixed to ensure that operations of `seed_seq::generate()` won't throw except from operations of the provided iterator range, but there is no corresponding "safety belt" for user-provided `SSeq` types, since 26.6.1.2 [rand.req.seedseq] does not impose no-throw requirements onto operations of *seed sequences*.

I. A quite radical step to fix this problem would be to impose general no-throw requirements on the expression `q.generate(rb,re)` from Table 115, but this is not as simple as it looks initially, because this function again depends on general types that are mutable random access iterators. Typically, we do not impose no-throw requirements on iterator operations and this would restrict general seed sequences where exceptions are not a problem. Furthermore, we do not impose comparable constraints for other expressions, like that of the expression `g()` in Table 116 for good reasons, e.g. `random_device::operator()` explicitly states when it throws exceptions.

II. A less radical variant of the previous suggestion would be to add a normative requirement on the expression `q.generate(rb,re)` from Table 115 that says: "Throws nothing if operations of `rb` and `re` do not throw exceptions". Nevertheless we typically do not describe *conditional* Throws elements in proper requirement sets elsewhere (Container requirements excluded, they just describe the containers from Clause 23) and this may exclude resonable implementations of seed sequences that could throw exceptions under rare situations.

III. The iterator arguments provided to `SSeq::generate()` for operations in templates of 26.6.3 [rand.eng] and 26.6.4 [rand.adapt]

are under control of implementations, so we could impose stricter exceptions requirements on `SSeq::generate()` for `SSeq` types that are used to instantiate member templates in 26.6.3 [rand.eng] and 26.6.4 [rand.adapt] solely.

IV. We simply add extra wording to the introductive parts of 26.6.3 [rand.eng] and 26.6.4 [rand.adapt] that specify that operations of the engine (adaptor) templates that depend on a template parameter `SSeq` throw no exception unless `SSeq::generate()` throws an exception.

Given these options I would suggest to apply the variant described in the fourth bullet.

The proposed resolution attempts to reduce a lot of the redundancies of requirements in the introductory paragraphs of 26.6.3 [rand.eng] and 26.6.4 [rand.adapt] by introducing a new intermediate sub-clause "Engine and engine adaptor class templates" following sub-clause 26.6.2 [rand.synopsis]. This approach also solves the problem that currently 26.6.3 [rand.eng] also describes requirements that apply for 26.6.4 [rand.adapt] (Constrained templates involving the `Sseq` parameters).

[2013-04-20, Bristol]

Remove the first bullet point:

> ?- Throughout this sub-clause general requirements and conventions are described that apply to every class template specified in sub-clause 26.6.3 [rand.eng] and 26.6.4 [rand.adapt]. Phrases of the form "in those sub-clauses" shall be interpreted as equivalent to "in sub-clauses 26.6.3 [rand.eng] and 26.6.4 [rand.adapt]".

Replace "in those sub-clauses" with "in sub-clauses 26.6.3 [rand.eng] and 26.6.4 [rand.adapt]".

Find another place for the wording.

Daniel: These are requirements on the implementation not on the types. I'm not comfortable in moving it to another place without double checking.

Improve the text (there are 4 "for"s): *for* copy constructors, *for* copy assignment operators, *for* streaming operators, and *for* equality and inequality operators are not shown in the synopses.

Move the information of this paragraph to the paragraphs it refers to:

> "-?- Descriptions are provided in those sub-clauses only for engine operations that are not described in 26.6.1.4 [rand.req.eng], for adaptor operations that are not described in 26.6.1.5 [rand.req.adapt], or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses."

Alisdair: I prefer duplication here than consolidation/reference to these paragraphs.

The room showed weakly favjust or for duplication.

Previous resolution from Daniel [SUPERSEDED]:

1. Add a new sub-clause titled "Engine and engine adaptor class templates" following sub-clause 26.6.2 [rand.synopsis] (but at the same level) and add one further sub-clause "General" as child of the new sub-clause as follows:

   Engine and engine adaptor class templates [rand.engadapt]

   General [rand.engadapt.general]

   -?- Throughout this sub-clause general requirements and conventions are described that apply to every class template specified in sub-clause 26.6.3 [rand.eng] and 26.6.4 [rand.adapt]. Phrases of the form "in those sub-clauses" shall be interpreted as equivalent to "in sub-clauses 26.6.3 [rand.eng] and 26.6.4 [rand.adapt]".

   -?- Except where specified otherwise, the complexity of each function specified in those sub-clauses is constant.

   -?- Except where specified otherwise, no function described in those sub-clauses throws an exception.

   -?- Every function described in those sub-clauses that has a function parameter `q` of type `SSeq&` for a template type parameter named `SSeq` that is different from type `std::seed_seq` throws what and when the invocation of `q.generate` throws.

-?- Descriptions are provided in those sub-clauses only for engine operations that are not described in 26.6.1.4 [rand.req.eng], for adaptor operations that are not described in 26.6.1.5 [rand.req.adapt], or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses.

-?- Each template specified in those sub-clauses requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

-?- For every random number engine and for every random number engine adaptor `X` defined in those sub-clauses:

- if the constructor

  ```
  template <class Sseq> explicit X(Sseq& q);
  ```

  is called with a type `Sseq` that does not qualify as a seed sequence, then this constructor shall not participate in overload resolution;

- if the member function

  ```
  template <class Sseq> void seed(Sseq& q);
  ```

  is called with a type `Sseq` that does not qualify as a seed sequence, then this function shall not participate in overload resolution;

The extent to which an implementation determines that a type cannot be a seed sequence is unspecified, except that as a minimum a type shall not qualify as a seed sequence if it is implicitly convertible to `X::result_type`.

2. Edit the contents of sub-clause 26.6.3 [rand.eng] as indicated:

-1- Each type instantiated from a class template specified in this section 26.6.3 [rand.eng] satisfies the requirements of a random number engine (26.6.1.4 [rand.req.eng]) type and the general implementation requirements specified in sub-clause [rand.engadapt.general].

~~-2- Except where specified otherwise, the complexity of each function specified in this section 26.6.3 [rand.eng] is constant.~~

~~-3- Except where specified otherwise, no function described in this section 26.6.3 [rand.eng] throws an exception.~~

~~-4- Descriptions are provided in this section 26.6.3 [rand.eng] only for engine operations that are not described in 26.6.1.4 [rand.req.eng] [...]~~

~~-5- Each template specified in this section 26.6.3 [rand.eng] requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. [...]~~

~~-6- For every random number engine and for every random number engine adaptor `x` defined in this subclause (26.6.3 [rand.eng]) and in sub-clause 26.6.3 [rand.eng]: [...]~~

3. Edit the contents of sub-clause 26.6.4.1 [rand.adapt.general] as indicated:

-1- Each type instantiated from a class template specified in this section ~~26.6.3 [rand.eng]~~26.6.4 [rand.adapt] satisfies the requirements of a random number engine adaptor (26.6.1.5 [rand.req.adapt]) type and the general implementation requirements specified in sub-clause [rand.engadapt.general].

~~-2- Except where specified otherwise, the complexity of each function specified in this section 26.6.4 [rand.adapt] is constant.~~

~~-3- Except where specified otherwise, no function described in this section 26.6.4 [rand.adapt] throws an exception.~~

~~-4- Descriptions are provided in this section 26.6.4 [rand.adapt] only for engine operations that are not described in 26.6.1.5 [rand.req.adapt] [...]~~

~~-5- Each template specified in this section 26.6.4 [rand.adapt] requires one or more relationships,~~

~~involving the value(s) of its non-type template parameter(s), to hold. [...]~~

[2014-02-09, Daniel provides alternative resolution]

[Lenexa 2015-05-07: Move to Ready]

LWG 2181 exceptions from seed sequence operations

STL: Daniel explained that I was confused. I said, oh, seed_seq says it can throw if the RanIt throws. Daniel says the RanIts are provided by the engine. Therefore if you give a seed_seq to an engine, it cannot throw, as implied by the current normative text. So what Daniel has in the PR is correct, if slightly unnecessary. It's okay to have explicitly non-overlapping Standardese even if overlapping would be okay.

Marshall: And this is a case where the std:: on seed_seq is a good thing.

STL: Meh.

STL: And that was my only concern with this PR. I like the latest PR much better than the previous.

Marshall: Yes. There's a drive-by fix for referencing the wrong section. Other than that, the two are the same.

STL: Alisdair wanted the repetition instead of centralization, and I agree.

Marshall: Any other opinions?

Jonathan: I'll buy it.

STL: For a dollar?

Hwrd: I'll buy that for a nickel.

Marshall: Any objections to Ready? I don't see a point in Immediate.

Jonathan: Absolutely agree.

Marshall: 7 for ready, 0 opposed, 0 abstain.

[2014-05-22, Daniel syncs with recent WP]

[2015-10-31, Daniel comments and simplifies suggested wording changes]

Upon Walter Brown's suggestion the revised wording does not contain any wording changes that could be considered as editorial.

Previous resolution from Daniel [SUPERSEDED]:

This wording is relative to N3936.

1. Edit the contents of sub-clause 26.6.3 [rand.eng] as indicated:

> -1- Each type instantiated from a class template specified in this section 26.6.3 [rand.eng] satisfies the requirements of a random number engine (26.6.1.4 [rand.req.eng]) type.
>
> -2- Except where specified otherwise, the complexity of each function specified in this section 26.6.3 [rand.eng] is constant.
>
> -3- Except where specified otherwise, no function described in this section 26.6.3 [rand.eng] throws an exception.
>
> -?- Every function described in this section 26.6.3 [rand.eng] that has a function parameter `q` of type `Sseq&` for a template type parameter named `Sseq` that is different from type `std::seed_seq` throws what and when the invocation of `q.generate` throws.
>
> -4- Descriptions are provided in this section 26.6.3 [rand.eng] only for engine operations that are not described in 26.6.1.4 [rand.req.eng] or for operations where there is additional semantic information. In particular, declarations for copy constructors, ~~for~~ copy assignment operators, ~~for~~ streaming operators, ~~and for~~ equality operators, and inequality operators are not shown in the synopses.

-5- Each template specified in this section 26.6.3 [rand.eng] requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

-6- For every random number engine and for every random number engine adaptor `X` defined in this subclause (26.6.3 [rand.eng]) and in sub-clause 26.6.4 [rand.adapt]:

- if the constructor

```
template <class Sseq> explicit X(Sseq& q);
```

  is called with a type `Sseq` that does not qualify as a seed sequence, then this constructor shall not participate in overload resolution;

- if the member function

```
template <class Sseq> void seed(Sseq& q);
```

  is called with a type `Sseq` that does not qualify as a seed sequence, then this function shall not participate in overload resolution;

The extent to which an implementation determines that a type cannot be a seed sequence is unspecified, except that as a minimum a type shall not qualify as a seed sequence if it is implicitly convertible to `X::result_type`.

2. Edit the contents of sub-clause 26.6.4.1 [rand.adapt.general] as indicated:

-1- Each type instantiated from a class template specified in this section ~~26.6.3 [rand.eng]~~26.6.4 [rand.adapt] satisfies the requirements of a random number engine adaptor (26.6.1.5 [rand.req.adapt]) type.

-2- Except where specified otherwise, the complexity of each function specified in this section 26.6.4 [rand.adapt] is constant.

-3- Except where specified otherwise, no function described in this section 26.6.4 [rand.adapt] throws an exception.

-?- Every function described in this section 26.6.4 [rand.adapt] that has a function parameter `q` of type `Sseq&` for a template type parameter named `Sseq` that is different from type `std::seed_seq` throws what and when the invocation of `q.generate` throws.

-4- Descriptions are provided in this section 26.6.4 [rand.adapt] only for adaptor operations that are not described in section 26.6.1.5 [rand.req.adapt] or for operations where there is additional semantic information. In particular, declarations for copy constructors, ~~for~~ copy assignment operators, ~~for~~ streaming operators, ~~and for~~ equality operators, and inequality operators are not shown in the synopses.

-5- Each template specified in this section 26.6.4 [rand.adapt] requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

3. Edit the contents of sub-clause 26.6.8.1 [rand.dist.general] p2 as indicated: [*Drafting note*: These editorial changes are just for consistency with those applied to 26.6.3 [rand.eng] and 26.6.4.1 [rand.adapt.general] — *end drafting note*]

-2- Descriptions are provided in this section 26.6.8 [rand.dist] only for distribution operations that are not described in 26.6.1.6 [rand.req.dist] or for operations where there is additional semantic information. In particular, declarations for copy constructors, ~~for~~ copy assignment operators, ~~for~~ streaming operators, ~~and for~~ equality operators, and inequality operators are not shown in the synopses.

Proposed resolution:

This wording is relative to N4527.

1. Edit the contents of sub-clause 26.6.3 [rand.eng] as indicated:

-1- [...]

-2- Except where specified otherwise, the complexity of each function specified in this section 26.6.3 [rand.eng] is constant.

-3- Except where specified otherwise, no function described in this section 26.6.3 [rand.eng] throws an exception.

-?- Every function described in this section 26.6.3 [rand.eng] that has a function parameter `q` of type `Sseq&` for a template type parameter named `Sseq` that is different from type `std::seed_seq` throws what and when the invocation of `q.generate` throws.

[...]

2. Edit the contents of sub-clause 26.6.4.1 [rand.adapt.general] as indicated:

-1- [...]

-2- Except where specified otherwise, the complexity of each function specified in this section 26.6.4 [rand.adapt] is constant.

-3- Except where specified otherwise, no function described in this section 26.6.4 [rand.adapt] throws an exception.

-?- Every function described in this section 26.6.4 [rand.adapt] that has a function parameter `q` of type `Sseq&` for a template type parameter named `Sseq` that is different from type `std::seed_seq` throws what and when the invocation of `q.generate` throws.

2309. `mutex::lock()` should not throw `device_or_resource_busy`

Section: 30.4.1.2 [thread.mutex.requirements.mutex] **Status:** Ready **Submitter:** Detlef Vollmann **Opened:** 2013-09-27 **Last modified:** 2016-03-06

Priority: 0

View all other issues in [thread.mutex.requirements.mutex].

View all issues with Ready status.

Discussion:

As discussed during the Chicago meeting in SG1 the only reasonable reasons for throwing `device_or_resource_busy` seem to be:

- The thread currently already holds the mutex, the mutex is not recursive, and the implementation detects this. In this case `resource_deadlock_would_occur` should be thrown.
- Priority reasons. At least `std::mutex` (and possibly all standard mutex types) should not be setup this way, otherwise we have real problems with `condition_variable::wait()`.

[2014-06-17 Rapperswil]

Detlef provides wording

[2015-02 Cologne]

Handed over to SG1.

[2015-05 Lenexa, SG1 response]

We believe we were already done with it. Should be in SG1-OK status.

[2015-10 pre-Kona]

SG1 hands this over to LWG for wording review

[2015-10 Kona]

Geoffrey provides new wording.

Previous resolution [SUPERSEDED]:

This wording is relative to N3936.

1. Change 30.4.1.2 [thread.mutex.requirements.mutex] as indicated:

   -13- *Error conditions*:

   - `operation_not_permitted` — if the thread does not have the privilege to perform the operation.
   - `resource_deadlock_would_occur` — if the implementation detects that a deadlock would occur.
   - ~~`device_or_resource_busy` — if the mutex is already locked and blocking is not possible.~~

Proposed resolution:

This wording is relative to N4527.

1. Change 30.4.1.2 [thread.mutex.requirements.mutex] as indicated:

   [...]

   -4- The error conditions for error codes, if any, reported by member functions of the mutex types shall be:

   (4.1) — `resource_unavailable_try_again` — if any native handle type manipulated is not available.

   (4.2) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

   ~~(4.3) — `device_or_resource_busy` — if any native handle type manipulated is already locked.~~

   [...]

   -13- *Error conditions*:

   (13.1) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

   (13.2) — `resource_deadlock_would_occur` — if the implementation detects that a deadlock would occur.

   ~~(13.3) — `device_or_resource_busy` — if the mutex is already locked and blocking is not possible.~~

2. Change 30.4.1.4 [thread.sharedmutex.requirements] as indicated:

   [...]

   -10- *Error conditions*:

   (10.1) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

   (10.2) — `resource_deadlock_would_occur` — if the implementation detects that a deadlock would occur.

   ~~(10.3) — `device_or_resource_busy` — if the mutex is already locked and blocking is not possible.~~

   [...]

2310. Public *exposition only* member in `std::array`

Section: 23.3.7.1 [array.overview] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2013-09-30 **Last modified:** 2016-03-06

Priority: 4

View all other issues in [array.overview].

View all issues with Ready status.

Discussion:

23.3.7.1 [array.overview] shows `std::array` with an "exposition only" data member, `elems`.

The wording in 17.5.2.3 [objects.within.classes] that defines how "exposition only" is used says it applies to private members, but `std::array::elems` (or its equivalent) must be public in order for `std::array` to be an aggregate.

If the intention is that `std::array::elems` places requirements on the implementation to provide "equivalent external behavior" to a public array member, then 17.5.2.3 [objects.within.classes] needs to cover public members too, or some other form should be used in 23.3.7.1 [array.overview].

[Urbana 2014-11-07: Move to Open]

[Kona 2015-10: Link to 2516]

[2015-11-14, Geoffrey Romer provides wording]

[2016-02-04, Tim Song improves the P/R]

Instead of the build-in address-operator, `std::addressof` should be used.

[2016-03 Jacksonville]

Move to Ready.

Proposed resolution:

This wording is relative to N4567.

1. Edit 23.3.7.1 [array.overview] as indicated

   [...]

   -3- An array [...]

   ```
   namespace std {
     template <class T, size_t N>
     struct array {
       […]
       ~~T elems[N]; // exposition only~~
       […]
     };
   }
   ```

   ~~-4- [*Note*: The member variable `elems` is shown for exposition only, to emphasize that array is a class aggregate. The name `elems` is not part of array's interface. — *end note*]~~

2. Edit 23.3.7.5 [array.data] as follows:

   ```
   T* data() noexcept;
   const T* data() const noexcept;
   ```

   -1- *Returns*: ~~`elems`~~A pointer such that `[data(), data() + size())` is a valid range, and `data() == addressof(front())`.

2328. Rvalue stream extraction should use perfect forwarding

Section: 27.7.2.6 [istream.rvalue] **Status:** Ready **Submitter:** Stephan T. Lavavej **Opened:** 2013-09-21 **Last modified:** 2016-03-06

Priority: 3

View other active issues in [istream.rvalue].

View all other issues in [istream.rvalue].

View all issues with Ready status.

Discussion:

27.7.2.6 [istream.rvalue] declares `operator>>(basic_istream<charT, traits>&& is, T& x)`. However, 27.7.2.2.3 [istream::extractors]/7 declares `operator>>(basic_istream<charT,traits>& in, charT* s)`, plus additional overloads for `unsigned char*` and `signed char*`. This means that "`return_rvalue_istream() >> &arr[0]`" won't compile, because `T&` won't bind to the rvalue `&arr[0]`.

[2014-02-12 Issaquah : recategorize as P3]

Jonathan Wakely: Bill was certain the change is right, I think so with less certainty

Jeffrey Yaskin: I think he's right, hate that we need this

Jonathan Wakely: is this the security issue Jeffrey raised on lib reflector?

Move to P3

[2015-05-06 Lenexa: Move to Review]

WEB, MC: Proposed wording changes one signature (in two places) to take a forwarding reference.

TK: Should be consistent with an istream rvalue?

MC: This is the case where you pass the stream by rvalue reference.

RP: I would try it before standardizing.

TK: Does it break anything?

RP, TK: It will take all arguments, will be an exact match for everything.

RS, TK: This adapts streaming into an rvalue stream to make it act like streaming into an lvalue stream.

RS: Should this really return the stream by lvalue reference instead of by rvalue reference? ⇒ new LWG issue.

RP: Security issue?

MC: No. That's `istream >> char*`, C++ version of `gets()`. Remove it, as we did for `gets()`? ⇒ new LWG issue.

RS: Proposed resolution looks correct to me.

MC: Makes me (and Jonathan Wakely) feel uneasy.

Move to Review, consensus.

[2016-01-15, Daniel comments and suggests improved wording]

It has been pointed out by Tim Song, that the initial P/R (deleting the *Returns* paragraph and making the *Effects* "equivalent to `return is >> /* stuff */;`") breaks cases where the applicable `operator>>` doesn't return a type that is convertible to a reference to the stream:

```
struct A{};
void operator>>(std::istream&, A&){ }

void f() {
  A a;
  std::istringstream() >> a; // was OK, now error
}
```

This seems like an unintended wording artifact of the "Equivalent to" Phrase Of Power and could be easily fixed by changing the second part of the P/R slightly as follows:

```
template <class charT, class traits, class T>
  basic_istream<charT, traits>&
  operator>>(basic_istream<charT, traits>&& is, T&& x);
```

-1- *Effects:* Equivalent to:~~is >>x~~

```
is >> std::forward<T>(x);
return is;
```

~~-2- *Returns:* is~~

Previous resolution [SUPERSEDED]:

This wording is relative to N4567.

1. Edit 27.7.1 [iostream.format.overview], header `<istream>` synopsis, as indicated:

```
namespace std {
  […]
  template <class charT, class traits, class T>
    basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>&& is, T&& x);
```

```
}
```

2. Edit 27.7.2.6 [istream.rvalue] as indicated:

```
template <class charT, class traits, class T>
  basic_istream<charT, traits>&
  operator>>(basic_istream<charT, traits>&& is, T&& x);
```

-1- *Effects:* Equivalent to `return is >>x std::forward<T>(x)`

~~-2- *Returns:* `is`~~

[2016-03 Jacksonville: Move to Ready]

Proposed resolution:

This wording is relative to N4567.

1. Edit 27.7.1 [iostream.format.overview], header `<istream>` synopsis, as indicated:

```
namespace std {
  […]
  template <class charT, class traits, class T>
    basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>&& is, T&& x);
}
```

2. Edit 27.7.2.6 [istream.rvalue] as indicated:

```
template <class charT, class traits, class T>
  basic_istream<charT, traits>&
  operator>>(basic_istream<charT, traits>&& is, T&& x);
```

-1- *Effects:* Equivalent to:~~`is >>x`~~

```
is >> std::forward<T>(x);
return is;
```

~~-2- *Returns:* `is`~~

2393. `std::function`'s *Callable* definition is broken

Section: 20.12.12.2 [func.wrap.func] **Status:** Ready **Submitter:** Daniel Krügler **Opened:** 2014-06-03 **Last modified:** 2016-03-06

Priority: 2

View other active issues in [func.wrap.func].

View all other issues in [func.wrap.func].

View all issues with Ready status.

Discussion:

The existing definition of `std::function`'s *Callable* requirements provided in 20.12.12.2 [func.wrap.func] p2,

> A callable object `f` of type `F` is *Callable* for argument types `ArgTypes` and return type `R` if the expression *`INVOKE`*`(f, declval<ArgTypes>()..., R)`, considered as an unevaluated operand (Clause 5), is well formed (20.9.2).

is defective in several aspects:

I. The wording can be read to be defined in terms of callable objects, not of callable types.

II. Contrary to that, 20.12.12.2.5 [func.wrap.func.targ] p2 speaks of "`T` shall be a type that is *Callable* (20.9.11.2) for parameter types `ArgTypes` and return type `R`."

III. The required value category of the callable object during the call expression (lvalue or rvalue) strongly depends on an interpretation of the expression `f` and therefore needs to be specified unambiguously.

The intention of original proposal (see IIIa. Relaxation of target requirements) was to refer to both types and values ("we say that the function object `f` (and its type `F`) is *Callable* […]"), but that mental model is not really deducible from the existing wording. An

improved type-dependence wording would also make the sfinae-conditions specified in 20.12.12.2.1 [func.wrap.func.con] p8 and p21 ("[...] shall not participate in overload resolution unless `f` is Callable (20.9.11.2) for argument types `ArgTypes...` and return type `R`.") easier to interpret.

My understanding always had been (see e.g. Howard's code example in the 2009-05-01 comment in LWG 815), that `std::function` invokes the call operator of its target via an **lvalue**. The required value-category is relevant, because it allows to reflect upon whether an callable object such as

```
struct RVF
{
  void operator()() const && {}
};
```

would be a feasible target object for `std::function<void()>` or not.

Clarifying the current *Callable* definition seems also wise to make a future transition to language-based concepts easier. A local fix of the current wording is simple to achieve, e.g. by rewriting it as follows:

> A callable ~~object `f` of~~ type (20.12.1 [func.def]) `F` is *Callable* for argument types `ArgTypes` and return type `R` if the expression *INVOKE*(~~`f`~~`declval<F&>()`, `declval<ArgTypes>()..., R`), considered as an unevaluated operand (Clause 5), is well formed (20.9.2).

It seems appealing to move such a general *Callable* definition to a more "fundamental" place (e.g. as another paragraph of 20.12.1 [func.def]), but the question arises, whether such a more general concept should impose the requirement that the call expression is invoked on an **lvalue** of the callable object — such a special condition would also conflict with the more general definition of the `result_of` trait, which is defined for either lvalues or rvalues of the callable type `Fn`. In this context I would like to point out that "*Lvalue-Callable*" is not the one and only *Callable* requirement in the library. Counter examples are `std::thread`, `call_once`, or `async`, which depend on "*Rvalue-Callable*", because they all act on functor rvalues, see e.g. 30.3.1.2 [thread.thread.constr]:

> [...] The new thread of execution executes *INVOKE*(*DECAY_COPY*(`std::forward<F>(f)`), *DECAY_COPY*(`std::forward<Args>(args))...`) [...]

For every callable object `F`, the result of *DECAY_COPY* is an rvalue. These implied rvalue function calls are no artifacts, but had been deliberately voted for by a Committee decision (see LWG 2021, 2011-06-13 comment) and existing implementations respect these constraints correctly. Just to give an example,

```
#include <thread>

struct LVF
{
  void operator()() & {}
};

int main()
{
  LVF lf;
  std::thread t(lf);
  t.join();
}
```

is supposed to be rejected.

The below presented wording changes are suggested to be minimal (still local to `std::function`), but the used approach would simplify a future (second) conceptualization or any further generalization of *Callable* requirements of the Library.

[2015-02 Cologne]

Related to N4348. Don't touch with a barge pole.

[2015-09 Telecom]

N4348 not going anywhere, can now touch with or without barge poles
Ville: where is Lvalue-Callable defined?
Jonathan: this is the definition. It's replacing Callable with a new term and defining that. Understand why it's needed, hate the change.
Geoff: punt to an LWG discussion in Kona

[2015-10 Kona]

STL: I like this in general. But we also have an opportunity here to add a precondition. By adding static assertions, we can make implementations better. Accept the PR but reinstate the requirement.

MC: Status Review, to be moved to TR at the next telecon.

[2015-10-28 Daniel comments and provides alternative wording]

The wording has been changed as requested by the Kona result. But I would like to provide the following counter-argument for this changed resolution: Currently the following program is accepted by three popular Standard libraries, Visual Studio 2015, gcc 6 libstdc++, and clang 3.8.0 libc++:

```
#include <functional>
#include <iostream>
#include <typeinfo>
#include "boost/function.hpp"

void foo(int) {}

int main() {
  std::function<void(int)> f(foo);
  std::cout << f.target<double>() << std::endl;
  boost::function<void(int)> f2(foo);
  std::cout << f2.target<double>() << std::endl;
}
```

and outputs the implementation-specific result for **two null pointer** values.

Albeit this code is not conforming, it is probable that similar code exists in the wild. The current boost documentation does not indicate *any* precondition for calling the `target` function, so it is natural that programmers would expect similar specification and behaviour.

Standardizing the suggested change requires a change of all implementations and I don't see any advantage for the user. With that change previously working code could now cause instantiation errors, I don't see how this could be considered as an improvement of the status quo. The result value of `target` is always a pointer, so a null-check by the user code is already required, therefore I really see no reason what kind of problem could result out of the current implementation behaviour, since the implementation never is required to perform a C cast to some funny type.

Previous resolution [SUPERSEDED]:

This wording is relative to N3936.

1. Change 20.12.12.2 [func.wrap.func] p2 as indicated:

   -2- A callable ~~object `f` of~~ type (20.12.1 [func.def]) `F` is *Lvalue-Callable* for argument types `ArgTypes` and return type `R` if the expression *INVOKE*(~~`f`~~`declval<F&>()`, `declval<ArgTypes>()...`, `R`), considered as an unevaluated operand (Clause 5), is well formed (20.9.2).

2. Change 20.12.12.2.1 [func.wrap.func.con] p8+p21 as indicated:

   ```
   template<class F> function(F f);
   template <class F, class A> function(allocator_arg_t, const A& a, F f);
   ```

   [...]

   -8- *Remarks*: These constructors shall not participate in overload resolution unless ~~`f`~~`F` is `Lvalue-Callable` (20.9.11.2) for argument types `ArgTypes...` and return type `R`.

   [...]

   ```
   template<class F> function& operator=(F&& f);
   ```

   [...]

   -21- *Remarks*: This assignment operator shall not participate in overload resolution unless ~~`declval<typename decay<F>::type&>()`~~`decay_t<F>` is `Lvalue-Callable` (20.9.11.2) for argument types `ArgTypes...` and return type `R`.

3. Change 20.12.12.2.5 [func.wrap.func.targ] p2 as indicated: [*Editorial comment*: Instead of adapting the preconditions for the naming change I recommend to strike it completely, because the `target()` functions do not depend on it; the corresponding wording exists since its initial proposal and it seems without any

advantage to me. Assume that some template argument `T` is provided, which does *not* satisfy the requirements: The effect will be that the result is a null pointer value, but that case can happen in other (valid) situations as well. — *end comment*]

```
template<class T> T* target() noexcept;
template<class T> const T* target() const noexcept;
```

~~-2- *Requires*: `T` shall be a type that is `Callable` (20.9.11.2) for parameter types `ArgTypes` and return type `R`.~~

-3- *Returns*: If `target_type() == typeid(T)` a pointer to the stored function target; otherwise a null pointer.

[2015-10, Kona Saturday afternoon]

GR explains the current short-comings. There's no concept in the standard that expresses rvalue member function qualification, and so, e.g. std::function cannot be forbidden from wrapping such functions. TK: Although it wouldn't currently compile.

GR: Implementations won't change as part of this. We're just clearing up the wording.

STL: I like this in general. But we also have an opportunity here to add a precondition. By adding static assertions, we can make implementations better. Accept the PR but reinstate the requirement.

JW: I hate the word "Lvalue-Callable". I don't have a better suggestion, but it'd be terrible to teach. AM: I like the term. I don't like that we need it, but I like it. AM wants the naming not to get in the way with future naming. MC: We'll review it.

TK: Why don't we also add Rvalue-Callable? STL: Because nobody consumes it.

Discussion whether "tentatively ready" or "review". The latter would require one more meeting. EF: We already have implementation convergence. MC: I worry about a two-meeting delay. WEB: All that being said, I'd be slightly more confident with a review since we'll have new wording, but I wouldn't object. MC: We can look at it in a telecon and move it.

STL reads out email to Daniel.

Status Review, to be moved to TR at the next telecon.

[2016-01-31, Daniel comments and suggests less controversive resolution]

It seems that specifically the wording changes for 20.12.12.2.5 [func.wrap.func.targ] p2 prevent this issue from making make progress. Therefore the separate issue LWG 2591 has been created, that focuses solely on this aspect. Furtheron the current P/R of this issue has been adjusted to the minimal possible one, where the term "Callable" has been replaced by the new term "Lvalue-Callable".

Previous resolution II [SUPERSEDED]:

This wording is relative to N4527.

1. Change 20.12.12.2 [func.wrap.func] p2 as indicated:

   -2- A callable ~~object `f` of~~ type (20.12.1 [func.def]) `F` is *Lvalue-Callable* for argument types `ArgTypes` and return type `R` if the expression *INVOKE*(~~`f`~~`declval<F&>()`, `declval<ArgTypes>()...`, `R`), considered as an unevaluated operand (Clause 5), is well formed (20.9.2).

2. Change 20.12.12.2.1 [func.wrap.func.con] p8+p21 as indicated:

   ```
   template<class F> function(F f);
   template <class F, class A> function(allocator_arg_t, const A& a, F f);
   ```

   [...]

   -8- *Remarks*: These constructors shall not participate in overload resolution unless ~~`f`~~`F` is `Lvalue-Callable` (20.9.11.2) for argument types `ArgTypes...` and return type `R`.

   [...]

   ```
   template<class F> function& operator=(F&& f);
   ```

   [...]

-21- *Remarks*: This assignment operator shall not participate in overload resolution unless ~~declval<typename decay<F>::type&>()~~decay_t<F> is Lvalue-Callable (20.9.11.2) for argument types ArgTypes... and return type R.

3. Change 20.12.12.2.5 [func.wrap.func.targ] p2 as indicated:

```
template<class T> T* target() noexcept;
template<class T> const T* target() const noexcept;
```

-2- *Remarks*: If T is a type that is not Lvalue-Callable (20.9.11.2) for parameter types ArgTypes and return type R, the program is ill-formed~~*Requires*: T shall be a type that is Callable (20.9.11.2) for parameter types ArgTypes and return type R~~.

-3- *Returns*: If target_type() == typeid(T) a pointer to the stored function target; otherwise a null pointer.

[2016-03 Jacksonville]

Move to Ready.

Proposed resolution:

This wording is relative to N4567.

1. Change 20.12.12.2 [func.wrap.func] p2 as indicated:

-2- A callable ~~object f of~~ type (20.12.1 [func.def]) F is *Lvalue-Callable* for argument types ArgTypes and return type R if the expression *INVOKE*(~~f~~declval<F&>(), declval<ArgTypes>()..., R), considered as an unevaluated operand (Clause 5), is well formed (20.9.2).

2. Change 20.12.12.2.1 [func.wrap.func.con] p8+p21 as indicated:

```
template<class F> function(F f);
template <class F, class A> function(allocator_arg_t, const A& a, F f);
```

[...]

-8- *Remarks*: These constructors shall not participate in overload resolution unless ~~f~~F is Lvalue-Callable (20.12.12.2 [func.wrap.func]) for argument types ArgTypes... and return type R.

[...]

```
template<class F> function& operator=(F&& f);
```

[...]

-21- *Remarks*: This assignment operator shall not participate in overload resolution unless ~~declval<typename decay<F>::type&>()~~decay_t<F> is Lvalue-Callable (20.12.12.2 [func.wrap.func]) for argument types ArgTypes... and return type R.

3. Change 20.12.12.2.5 [func.wrap.func.targ] p2 as indicated:

```
template<class T> T* target() noexcept;
template<class T> const T* target() const noexcept;
```

-2- *Requires*: T shall be a type that is Lvalue-Callable (20.12.12.2 [func.wrap.func]) for parameter types ArgTypes and return type R.

-3- *Returns*: If target_type() == typeid(T) a pointer to the stored function target; otherwise a null pointer.

2426. Issue about `compare_exchange`

Section: 29.6.5 [atomics.types.operations.req] **Status:** Ready **Submitter:** Hans Boehm **Opened:** 2014-08-25 **Last modified:** 2016-03-06

Priority: 1

View other active issues in [atomics.types.operations.req].

View all other issues in [atomics.types.operations.req].

View all issues with Ready status.

Discussion:

The standard is either ambiguous or misleading about the nature of accesses through the `expected` argument to the `compare_exchange_*` functions in 29.6.5 [atomics.types.operations.req]p21.

It is unclear whether the access to `expected` is itself atomic (intent clearly no) and exactly when the implementation is allowed to read or write it. These affect the correctness of reasonable code.

Herb Sutter, summarizing a complaint from Duncan Forster wrote:

> Thanks Duncan,
>
> I think we have a bug in the standardese wording and the implementations are legal, but let's check with the designers of the feature.
>
> Let me try to summarize the issue as I understand it:
>
> 1. What I think was intended: Lawrence, I believe you championed having `compare_exchange_*` take the `expected` value by reference, and update `expected` on failure to expose the old value, but this was only for convenience to simplify the calling loops which would otherwise always have to write an extra "reload" line of code. Lawrence, did I summarize your intent correctly?
>
> 2. What I think Duncan is trying to do: However, it turns out that, now that `expected` is an lvalue, it has misled(?) Duncan into trying to use the success of `compare_exchange_*` to hand off ownership *of `expected` itself* to another thread. For that to be safe, if the `compare_exchange_*` succeeds then the thread that performed it must no longer read or write from `expected` else his technique contains a race. Duncan, did I summarize your usage correctly? Is that the only use that is broken?
>
> 3. What the standard says: I can see why Duncan thinks the standard supports his use, but I don't think this was intended (I don't remember this being discussed but I may have been away for that part) and unless you tell me this was intended I think it's a defect in the standard. From 29.6.5 [atomics.types.operations.req]/21:
>
>   > -21- *Effects*: Atomically, compares the contents of the memory pointed to by `object` or by `this` for equality with that in `expected`, and if true, replaces the contents of the memory pointed to by `object` or by `this` with that in `desired`, and if false, updates the contents of the memory in `expected` with the contents of the memory pointed to by `object` or by `this`. [...]
>
>   I think we have a wording defect here in any case, because the "atomically" should not apply to the entire sentence — I'm pretty sure we never intended the atomicity to cover the write to `expected`.
>
>   As a case in point, borrowing from Duncan's mail below, I think the following implementation is intended to be legal:
>
> ```
> inline int _Compare_exchange_seq_cst_4(volatile _Uint4_t *_Tgt, _Uint4_t *_Exp, _Uint4_t _Value)
> { /* compare and exchange values atomically with
>      sequentially consistent memory order */
>   int _Res;
>   _Uint4_t _Prev = _InterlockedCompareExchange((volatile long *)_Tgt, _Value, *_Exp);
>   if (_Prev == *_Exp) //!!!!! Note the unconditional read from *_Exp here
>     _Res = 1;
>   else
>   { /* copy old value */
>     _Res = 0;
>     *_Exp = _Prev;
>   }
>   return (_Res);
> }
> ```
>
>   I think this implementation is intended to be valid — I think the only code that could be broken with the "!!!!!" read of `*_Exp` is Duncan's use of treating `a.compare_exchange_*(expected, desired) == true` as implying `expected` got handed off, because then another thread could validly be using `*_Exp` — but we never intended this use, right?

In a different thread Richard Smith wrote about the same problem:

> The `atomic_compare_exchange` functions are described as follows:
>
> > "Atomically, compares the contents of the memory pointed to by `object` or by `this` for equality with that in `expected`, and if true, replaces the contents of the memory pointed to by `object` or by `this` with that in `desired`, and if false, updates the contents of the memory in `expected` with the contents of the memory pointed to by `object` or by `this`. Further, if the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`."
>
> I think this is less clear than it could be about the effects of these operations on `*expected` in the failure case:
>
> 1. We have "Atomically, compares [...] and updates the contents of the memory in `expected` [...]". The update to the memory in `expected` is clearly not atomic, and yet this wording parallels the success case, in which the memory update is atomic.
> 2. The wording suggests that memory (including `*expected`) is affected according to the value of `failure`. In particular, the failure order could be `memory_order_seq_cst`, which might lead someone to incorrectly think they'd published the value of `*expected`.
>
> I think this can be clarified with no change in meaning by reordering the wording a little:
>
> > "Atomically, compares the contents of the memory pointed to by `object` or by `this` for equality with that in `expected`, and if true, replaces the contents of the memory pointed to by `object` or by `this` with that in `desired`~~, and if~~. If the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`. Further, if the comparison is false, ~~updates~~replaces the contents of the memory in `expected` with the ~~contents of~~value that was atomically read from the memory pointed to by `object` or by `this`. ~~Further, if the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`.~~"

Jens Maurer add:

> I believe this is an improvement.
>
> I like to see the following additional improvements:
>
> - "contents of the memory" is strange phrasing, which doesn't say how large the memory block is. Do we compare the values or the value representation of the lvalue `*object` (or `*this`)?
> - 29.3 [atomics.order] defines memory order based on the "affected memory location". It would be better to say something like "If the comparison is true, the memory synchronization order for the affected memory location `*object` is [...]"

There was also a discussion thread involving Herb Sutter, Hans Boehm, and Lawrence Crowl, resulting in proposed wording along the lines of:

> -21- *Effects*: Atomically with respect to `expected` and the memory pointed to by `object` or by `this`, compares the contents of the memory pointed to by `object` or by `this` for equality with that in `expected`, and if and only if true, replaces the contents of the memory pointed to by `object` or by `this` with that in desired, and if and only if false, updates the contents of the memory in expected with the contents of the memory pointed to by `object` or by `this`.
>
> At the end of paragraph 23, perhaps add
>
> > [*Example*: Because the `expected` value is updated only on failure, code releasing the memory containing the `expected` value on success will work. E.g. list head insertion will act atomically and not have a data race in the following code.
> >
> > ```
> > do {
> >   p->next = head; // make new list node point to the current head
> > } while(!head.compare_exchange_weak(p->next, p)); // try to insert
> > ```
> >
> > — *end example*]

Hans objected that this still gives the misimpression that the update to `expected` is atomic.

[2014-11 Urbana]

Proposed resolution was added after Redmond.

Recommendations from SG1:

1. Change wording to ~~if true~~if and only if true, and change ~~if false~~if and only if false.
2. If they want to add "respect to" clause, say "respect to object or this".
3. In example, load from head should be "head.load(memory_order_relaxed)", because people are going to use example as example of good code.

(wording edits not yet applied)

[2015-02 Cologne]

Handed over to SG1.

[2015-05 Lenexa, SG1 response]

We believed we were done with it, but it was kicked back to us, with the wording we suggested not yet applied. It may have been that our suggestions were unclear. Was that the concern?

[2016-02 Jacksonville]

Applied the other half of the "if and only if" response from SG1, and moved to Ready.

Proposed resolution:

1. Edit 29.6.5 [atomics.types.operations.req] p21 as indicated:

> -21- *Effects*: Retrieves the value in `expected`. It then a~~A~~tomically~~,~~ compares the contents of the memory pointed to by `object` or by `this` for equality with that ~~in~~previously retrieved from `expected`, and if true, replaces the contents of the memory pointed to by `object` or by `this` with that in `desired`~~, and if false, updates the contents of the memory in expected with the contents of the memory pointed to by `object` or by `this`. Further, if~~. If and only if the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`. If and only if the comparison is false then, after the atomic operation, the contents of the memory in `expected` are replaced by the value read from `object` or by `this` during the atomic comparison. If the operation returns true, these operations are atomic read-modify-write operations (1.10) on the memory pointed to by `this` or `object`. Otherwise, these operations are atomic load operations on that memory.

2. Add the following example to the end of 29.6.5 [atomics.types.operations.req] p23:

> -23- [*Note*: [...] — *end note*] [*Example*: [...] — *end example*]
>
> [*Example*: Because the expected value is updated only on failure, code releasing the memory containing the `expected` value on success will work. E.g. list head insertion will act atomically and would not introduce a data race in the following code:
>
> ```
> do {
>   p->next = head; // make new list node point to the current head
> } while(!head.compare_exchange_weak(p->next, p)); // try to insert
> ```
>
> — *end example*]

2436. Comparators for associative containers should always be `CopyConstructible`

Section: 23.2.4 [associative.reqmts], 23.2.5 [unord.req] **Status:** Ready **Submitter:** Stephan T. Lavavej **Opened:** 2014-10-01 **Last modified:** 2016-03-04

Priority: 2

View other active issues in [associative.reqmts].

View all other issues in [associative.reqmts].

View all issues with Ready status.

Discussion:

The associative container requirements attempt to permit comparators that are `DefaultConstructible` but non-`CopyConstructible`. However, the Standard contradicts itself. 23.4.4.1 [map.overview] depicts `map() : map(Compare()) { }` which requires both `DefaultConstructible` and `CopyConstructible`.

Unlike fine-grained element requirements (which are burdensome for implementers, but valuable for users), such fine-grained comparator requirements are both burdensome for implementers (as the Standard's self-contradiction demonstrates) and worthless for users. We should unconditionally require `CopyConstructible` comparators. (Note that `DefaultConstructible` should remain optional; this is not problematic for implementers, and allows users to use lambdas.)

Key equality predicates for unordered associative containers are also affected. However, 17.6.3.4 [hash.requirements]/1 already requires hashers to be `CopyConstructible`, so 23.2.5 [unord.req]'s redundant wording should be removed.

[2015-02, Cologne]

GR: I prefer to say "`Compare`" rather than "`X::key_compare`", since the former is what the user supplies. JY: It makes sense to use "`Compare`" when we talk about requirements but "`key_compare`" when we use it.

AM: We're adding requirements here, which is a breaking change, even though nobody will ever have had a non-`CopyConstructible` comparator. But the simplification is probably worth it.

GR: I don't care about unmovable containers. But I do worry that people might want to move they comparators. MC: How do you reconcile that with the function that says "give me the comparator"? GR: That one returns by value? JY: Yes. [To MC] You make it a requirement of that function. [To GR] And it [the `key_comp()` function] is missing its requirements. We need to add them everywhere. GR: map already has the right requirements.

JM: I dispute this. If in C++98 a type wasn't copyable, it had some interesting internal state, but in C++98 you wouldn't have been able to pass it into the container since you would have had to make a copy. JY: No, you could have default-constructed it and never moved it, e.g. a mutex. AM: So, it's a design change, but one that we should make. That's probably an LEWG issue. AM: There's a contradiction in the Standard here, and we need to fix it one way or another.

Conclusion: Move to LEWG

[2016-03, Jacksonville]

Adding `CopyConstructible` requirement OK.

Unanimous yes.

We discussed allowing `MoveConstructible`. A moved-from `set<>` might still contain elements, and using them would become undefined if the comparator changed behavior.

Proposed resolution:

This wording is relative to N3936.

1. Change 23.2.4 [associative.reqmts] Table 102 as indicated (Editorial note: For "expression" `X::key_compare` "defaults to" is redundant with the class definitions for map/etc.):

Table 102 — Associative container requirements

Expression	Return type	Assertion/note pre-/post-condition	Complexity
		...	
`X::key_compare`	`Compare`	~~defaults to `less<key_type>`~~ *Requires*: `key_compare` is `CopyConstructible`.	compile time
		...	
`X(c)` `X a(c);`		~~*Requires*: `key_compare` is `CopyConstructible`.~~ *Effects*: Constructs an empty container. Uses a copy of `c` as a comparison object.	constant
		...	
`X(i,j,c)`		*Requires*: ~~`key_compare` is `CopyConstructible`.~~	

`X a(i,j,c);`		`value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects*: Constructs [...]	[...]
...			

2. Change 23.2.5 [unord.req] Table 103 as indicated:

Table 103 — Unordered associative container requirements (in addition to container)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
...			
`X::key_equal`	`Pred`	*Requires*: `Pred` is `CopyConstructible`. `Pred` shall be a binary predicate that takes two arguments of type `Key`. `Pred` is an equivalence relation.	compile time
...			
`X(n, hf, eq)` `X a(n, hf, eq);`	`X`	~~*Requires:* `hasher` and `key_equal` are `CopyConstructible`.~~ *Effects*: Constructs [...]	[...]
...			
`X(n, hf)` `X a(n, hf);`	`X`	*Requires*: ~~`hasher` is `CopyConstructible` and~~ `key_equal` is `DefaultConstructible` *Effects*: Constructs [...]	[...]
...			
`X(i, j, n, hf, eq)` `X a(i, j, n, hf, eq);`	`X`	*Requires*: ~~`hasher` and `key_equal` are `CopyConstructible`.~~ `value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects*: Constructs [...]	[...]
...			
`X(i, j, n, hf)` `X a(i, j, n, hf);`	`X`	*Requires*: ~~`hasher` is `CopyConstructible` and~~ `key_equal` is `DefaultConstructible` `value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects*: Constructs [...]	[...]
...			

2441. Exact-width atomic typedefs should be provided

Section: 29.5 [atomics.types.generic] **Status:** Ready **Submitter:** Stephan T. Lavavej **Opened:** 2014-10-01 **Last modified:** 2016-03-06

Priority: 0

View other active issues in [atomics.types.generic].

View all other issues in [atomics.types.generic].

View all issues with Ready status.

Discussion:

`<atomic>` doesn't provide counterparts for `<inttypes.h>`'s most useful typedefs, possibly because they're quasi-optional. We can easily fix this.

[2014-11, Urbana]

Typedefs were transitional compatibility hack. Should use `_Atomic` macro or template. E.g. `_Atomic(int8_t)`. BUT `_Atomic` disappeared!

Detlef will look for `_Atomic` macro. If missing, will open issue.

[2014-11-25, Hans comments]

There is no `_Atomic` in C++. This is related to the much more general unanswered question of whether C++17 should reference C11, C99, or neither.

[2015-02 Cologne]

AM: I think this is still an SG1 issue; they need to deal with it before we do.

[2015-05 Lenexa, SG1 response]

Move to SG1-OK status. This seems like an easy short-term fix. We probably need a paper on C/C++ atomics compatibility to deal with `_Atomic`, but that's a separable issue.

[2015-10 pre-Kona]

SG1 hands this over to LWG for wording review

Proposed resolution:

This wording is relative to N3936.

1. Change 29.5 [atomics.types.generic] p8 as depicted:

   -8- There shall be atomic typedefs corresponding to the typedefs in the header `<inttypes.h>` as specified in Table 147. `atomic_intN_t`, `atomic_uintN_t`, `atomic_intptr_t`, and `atomic_uintptr_t` shall be defined if and only if `intN_t`, `uintN_t`, `intptr_t`, and `uintptr_t` are defined, respectively.

2. Change 29.6.5 [atomics.types.operations.req], Table 147 ("`atomic <inttypes.h>` typedefs"), as depicted:

Table 147 — `atomic <inttypes.h>` typedefs

Atomic typedef	`<inttypes.h>` type
`atomic_int8_t`	`int8_t`
`atomic_uint8_t`	`uint8_t`
`atomic_int16_t`	`int16_t`
`atomic_uint16_t`	`uint16_t`
`atomic_int32_t`	`int32_t`
`atomic_uint32_t`	`uint32_t`
`atomic_int64_t`	`int64_t`
`atomic_uint64_t`	`uint64_t`
...	

2451. [fund.ts.v2] `optional<T>` should 'forward' `T`'s implicit conversions

Section: 5.3 [fund.ts.v2::optional.object] **Status:** Ready **Submitter:** Geoffrey Romer **Opened:** 2014-10-31 **Last modified:** 2016-03-04

Priority: Not Prioritized

View other active issues in [fund.ts.v2::optional.object].

View all other issues in [fund.ts.v2::optional.object].

View all issues with Ready status.

Discussion:

Addresses: fund.ts.v2

Code such as the following is currently ill-formed (thanks to STL for the compelling example):

```
optional<string> opt_str = "meow";
```

This is because it would require two user-defined conversions (from `const char*` to `string`, and from `string` to `optional<string>`) where the language permits only one. This is likely to be a surprise and an inconvenience for users.

`optional<T>` should be implicitly convertible from any `U` that is implicitly convertible to `T`. This can be implemented as a non-explicit constructor template `optional(U&&)`, which is enabled via SFINAE only if `is_convertible_v<U, T>` and `is_constructible_v<T, U>`, plus any additional conditions needed to avoid ambiguity with other constructors (see N4064, particularly the "Odd" example, for why `is_convertible` and `is_constructible` are both needed; thanks to Howard Hinnant for spotting this).

In addition, we may want to support explicit construction from `U`, which would mean providing a corresponding explicit constructor with a complementary SFINAE condition (this is the single-argument case of the "perfect initialization" pattern described in N4064).

[2015-10, Kona Saturday afternoon]

STL: This has status LEWG, but it should be priority 1, since we cannot ship an IS without this.

TK: We assigned our own priorities to LWG-LEWG issues, but haven't actually processed any issues yet.

MC: This is important.

[2016-02-17, Ville comments and provides concrete wording]

I have prototype-implemented this wording in libstdc++. I didn't edit the copy/move-assignment operator tables into the new `operator=` templates that take `optional`s of a different type; there's a drafting note that suggests copying them from the existing tables.

[2016-03, Jacksonville]

Discussion of whether `variant` supports this. We think it does.

Take it for C++17.

Unanimous yes.

Proposed resolution:

This wording is relative to N4562.

1. Edit 5.3 [optional.object] as indicated:

```
template <class T>
class optional
{
public:
  typedef T value_type;

  // 5.3.1, Constructors
  constexpr optional() noexcept;
  constexpr optional(nullopt_t) noexcept;
  optional(const optional&);
  optional(optional&&) noexcept(see below);
  constexpr optional(const T&);
  constexpr optional(T&&);
  template <class... Args> constexpr explicit optional(in_place_t, Args&&...);
  template <class U, class... Args>
    constexpr explicit optional(in_place_t, initializer_list<U>, Args&&...);
  template <class U> constexpr optional(U&&);
  template <class U> constexpr optional(const optional<U>&);
  template <class U> constexpr optional(optional<U>&&);

  […]

  // 5.3.3, Assignment
  optional& operator=(nullopt_t) noexcept;
  optional& operator=(const optional&);
  optional& operator=(optional&&) noexcept(see below);
  template <class U> optional& operator=(U&&);
  template <class U> optional& operator=(const optional<U>&);
  template <class U> optional& operator=(optional<U>&&);
  template <class... Args> void emplace(Args&&...);
  template <class U, class... Args>
    void emplace(initializer_list<U>, Args&&...);

  […]

};
```

2. In 5.3.1 [optional.object.ctor], insert new signature specifications after p33:

   [*Note*: The following constructors are conditionally specified as `explicit`. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. — *end note*]

```
template <class U>
```

`constexpr optional(U&& v);`

-?- *Effects*: Initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `std::forward<U>(v)`.

-?- *Postconditions*: `*this` contains a value.

-?- *Throws*: Any exception thrown by the selected constructor of `T`.

-?- *Remarks*: If `T`'s selected constructor is a `constexpr` constructor, this constructor shall be a `constexpr` constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>` is `true` and `U` is not the same type as `T`. The constructor is `explicit` if and only if `is_convertible_v<U&&, T>` is `false`.

```
template <class U>
constexpr optional(const optional<U>& rhs);
```

-?- *Effects*: If `rhs` contains a value, initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `*rhs`.

-?- *Postconditions*: `bool(rhs) == bool(*this)`.

-?- *Throws*: Any exception thrown by the selected constructor of `T`.

-?- *Remarks*: If `T`'s selected constructor is a `constexpr` constructor, this constructor shall be a `constexpr` constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<T, const U&>` is `true`, `is_same<decay_t<U>, T>` is `false`, `is_constructible_v<T, const optional<U>&>` is `false` and `is_convertible_v<const optional<U>&, T>` is `false`. The constructor is `explicit` if and only if `is_convertible_v<const U&, T>` is `false`.

```
template <class U>
constexpr optional(optional<U>&& rhs);
```

-?- *Effects*: If `rhs` contains a value, initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `std::move(*rhs)`. `bool(rhs)` is unchanged.

-?- *Postconditions*: `bool(rhs) == bool(*this)`.

-?- *Throws*: Any exception thrown by the selected constructor of `T`.

-?- *Remarks*: If `T`'s selected constructor is a `constexpr` constructor, this constructor shall be a `constexpr` constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>` is `true`, `is_same<decay_t<U>, T>` is `false`, `is_constructible_v<T, optional<U>&&>` is `false` and `is_convertible_v<optional<U>&&, T>` is `false` and `U` is not the same type as `T`. The constructor is `explicit` if and only if `is_convertible_v<U&&, T>` is `false`.

3. In 5.3.3 [optional.object.assign], change as indicated:

```
template <class U> optional<T>& operator=(U&& v);
```

-22- *Remarks:* If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `v` is determined by the exception safety guarantee of `T`'s assignment. The function shall not participate in overload resolution unless `decay_t<U>` is not `nullopt_t` and `decay_t<U>` is not a specialization of `optional`~~`is_same_v<decay_t<U>, T> is true`~~.

~~-23- *Notes*: The reason for providing such generic assignment and then constraining it so that effectively `T == U` is to guarantee that assignment of the form `o = {}` is unambiguous.~~

```
template <class U> optional<T>& operator=(const optional<U>& rhs);
```

-?- *Requires*: `is_constructible_v<T, const U&>` is `true` and `is_assignable_v<T&, const U&>` is `true`.

-?- *Effects*:

Table ? — `optional::operator=(const optional<U>&)` effects

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `*rhs` to the contained value	initializes the contained value as if direct-non-list-initializing an object of type `T` with `*rhs`

rhs does not contain a value	destroys the contained value by calling `val->T::~T()`	no effect

-?- *Returns*: `*this`.

-?- *Postconditions*: `bool(rhs) == bool(*this)`.

-?- *Remarks*: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment. The function shall not participate in overload resolution unless `is_same_v<decay_t<U>, T>` is `false`.

`template <class U> optional<T>& operator=(optional<U>&& rhs);`

-?- *Requires*: `is_constructible_v<T, U>` is `true` and `is_assignable_v<T&, U>` is `true`.

-?- *Effects*: The result of the expression `bool(rhs)` remains unchanged.

Table ? — `optional::operator=(optional<U>&&)` effects

	***this contains a value**	***this does not contain a value**
rhs contains a value	assigns `std::move(*rhs)` to the contained value	initializes the contained value as if direct-non-list-initializing an object of type `T` with `std::move(*rhs)`
rhs does not contain a value	destroys the contained value by calling `val->T::~T()`	no effect

-?- *Returns*: `*this`.

-?- *Postconditions*: `bool(rhs) == bool(*this)`.

-?- *Remarks*: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment. The function shall not participate in overload resolution unless `is_same_v<decay_t<U>, T>` is `false`.

2509. [fund.ts.v2] `any_cast` doesn't work with rvalue reference targets and cannot move with a value target

Section: 6.4 [fund.ts.v2::any.nonmembers] **Status:** Tentatively Ready **Submitter:** Ville Voutilainen **Opened:** 2015-06-13 **Last modified:** 2016-04-15

Priority: 2

View all issues with Tentatively Ready status.

Discussion:

Addresses: fund.ts.v2

In Library Fundamentals v1, [any.nonmembers]/5 says:

> For the first form, `*any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand)`. For the second and third forms, `*any_cast<remove_reference_t<ValueType>>(&operand)`.

1. This means that

   ```
   any_cast<Foo&&>(whatever_kind_of_any_lvalue_or_rvalue);
   ```

   is always ill-formed. That's unfortunate, because forwarding such a cast result of an `any` is actually useful, and such uses do not want to copy/move the underlying value just yet.

2. Another problem is that that same specification prevents an implementation from moving to the target when

   ```
   ValueType any_cast(any&& operand);
   ```

is used. The difference is observable, so an implementation can't perform an optimization under the as-if rule. We are pessimizing every `CopyConstructible` *and* `MoveConstructible` type because we are not using the move when we can. This unfortunately includes types such as the library containers, and we do not want such a pessimization!

[2015-07, Telecom]

Jonathan to provide wording

[2015-10, Kona Saturday afternoon]

Eric offered to help JW with wording

Move to Open

[2016-01-30, Ville comments and provides wording]

Drafting note: the first two changes add support for types that have explicitly deleted move constructors. Should we choose not to support such types at all, the third change is all we need. For the second change, there are still potential cases where *Requires* is fulfilled but *Effects* is ill-formed, if a suitably concocted type is thrown into the mix.

Proposed resolution:

This wording is relative to N4562.

1. In 6.3.1 [any.cons] p11+p12, edit as follows:

```
template<class ValueType>
  any(ValueType&& value);
```

-10- Let `T` be equal to `decay_t<ValueType>`.

-11- *Requires*: `T` shall satisfy the `CopyConstructible` requirements, except for the requirements for `MoveConstructible`. If `is_copy_constructible_v<T>` is `false`, the program is ill-formed.

-12- *Effects*: If `is_constructible_v<T, ValueType&&>` is true, cConstructs an object of type `any` that contains an object of type `T` direct-initialized with `std::forward<ValueType>(value)`. Otherwise, constructs an object of type `any` that contains an object of type `T` direct-initialized with `value`.

[...]

2. In 6.4 [any.nonmembers] p5, edit as follows:

```
template<class ValueType>
  ValueType any_cast(const any& operand);
template<class ValueType>
  ValueType any_cast(any& operand);
template<class ValueType>
  ValueType any_cast(any&& operand);
```

-4- *Requires*: `is_reference_v<ValueType>` is `true` or `is_copy_constructible_v<ValueType>` is `true`. Otherwise the program is ill-formed.

-5- *Returns*: For the first form, `*any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand)`. For the second ~~and third~~ form~~s~~, `*any_cast<remove_reference_t<ValueType>>(&operand)`. For the third form, if `is_move_constructible_v<ValueType>` is `true` and `is_lvalue_reference_v<ValueType>` is `false`, `std::move(*any_cast<remove_reference_t<ValueType>>(&operand))`, otherwise, `*any_cast<remove_reference_t<ValueType>>(&operand)`.

[...]

2516. [fund.ts.v2] Public "exposition only" members in `observer_ptr`

Section: 8.12.1 [fund.ts.v2::memory.observer.ptr.overview] **Status:** Ready **Submitter:** Tim Song **Opened:** 2015-07-07 **Last modified:** 2016-03-06

Priority: 2

View all issues with Ready status.

Discussion:

Addresses: fund.ts.v2

In N4529 [memory.observer.ptr.overview], the public member typedefs `pointer` and `reference` are marked "//exposition-only".

Clause 17 of the standard, which the TS incorporates by reference, only defines "exposition only" for private members (see 17.5.2.3 [objects.within.classes], also compare LWG 2310). These types should be either made private, or not exposition only.

[2015-07, Telecom]

Geoffrey: Will survey "exposition-only" use and come up with wording to define "exposition-only" for data members, types, and concepts.

[2015-10, Kona Saturday afternoon]

GR: I cannot figure out what "exposition only" means. JW explained it to me once as meaning that "it has the same normative effect as if the member existed, without the effect of there actually being a member". But I don't know what is the "normative effect of the existence of a member" is.

MC: The "exposition-only" member should be private. GR: But it still has normative effect.

GR, STL: Maybe the member is public so it can be used in the public section? WEB (original worder): I don't know why the member is public. I'm not averse to changing it. GR: A telecon instructed me to find out how [exposition-only] is used elsewhere, and I came back with this issue because I don't understand what it means.

STL: We use exposition-only in many places. We all know what it means. GR: I don't know precisely what means.

STL: I know exactly what it means. STL suggests a rewording: the paragraph already says what we want, it's just a little too precise. GR not sure.

WEB: Are there any non-members in the Standard that are "exposition only"? STL: DECAY_COPY and INVOKE.

GR: There are a few enums.

HH: We have nested class types that are exposition-only. HH: We're about to make "none_t" an exposition-only type. [Everyone lists some examples from the Standard that are EO.] GR: "bitmask" contains some hypothetical types. That potentially contains user requirements (since regexp requires certain trait members to be "bitmask types").

STL: What's the priority of this issue? Does this have any effect on implementations? Is there anything mysterious that could happen?

AM: 2.

STL: How did that happen? It's at best priority 4. We have so many better things we could be doing.

WEB: "exposition only" in [???] is just used as plain English, not as words of power. GR: That gives me enough to make wording to fix this.

STL: I wouldn't mind if we didn't fix this ever.

MC: @JW, please write up wording for 2310 to make the EO members private. JW: I can't do that, because that'd make the class a non-aggregate.

WEB: Please cross-link both issues and move 2516 to "open".

[2015-11-14, Geoffrey Romer provides wording]

[2016-03 Jacksonville]

Move to ready. Note that this modifies both the Draft Standard and LFTS 2

Proposed resolution:

A. This part of the wording is against the working draft of the standard for Programming Language C++, the wording is relative to N4567.

   1. Edit 17.5.2.3 [objects.within.classes]/p2 as follows:

      -2- ~~Objects of certain classes are sometimes required by the external specifications of their classes to store data, apparently in member objects.~~ For the sake of exposition, some subclauses provide representative declarations, and semantic requirements, for private members ~~objects~~ of classes that meet the external

specifications of the classes. The declarations for such members ~~objects and the definitions of related member types~~ are followed by a comment that ends with *exposition only*, as in:

```
streambuf* sb; // exposition only
```

B. This part of the wording is against the working draft of the C++ Extensions for Library Fundamentals, Version 2, the wording is relative to N4529

1. Edit the synopsis in 8.12.1 [memory.observer.ptr.overview] as follows:

```
namespace std {
namespace experimental {
inline namespace fundamentals_v2 {

  template <class W> class observer_ptr {
    using pointer = add_pointer_t<W>;            // exposition-only
    using reference = add_lvalue_reference_t<W> // exposition-only
  public:
    // publish our template parameter and variations thereof
    using element_type = W;
    using pointer = add_pointer_t<W>;            // exposition-only
    using reference = add_lvalue_reference_t<W> // exposition-only

    // 8.12.2, observer_ptr constructors
    // default c'tor
    constexpr observer_ptr() noexcept;

    […]
  }; // observer_ptr<>

} // inline namespace fundamentals_v2
} // namespace experimental
} // namespace std
```

2542. Missing `const` requirements for associative containers

Section: 23.2.4 [associative.reqmts] **Status:** Ready **Submitter:** Daniel Krügler **Opened:** 2015-09-26 **Last modified:** 2016-03-06

Priority: 1

View other active issues in [associative.reqmts].

View all other issues in [associative.reqmts].

View all issues with Ready status.

Discussion:

Table 101 — "Associative container requirements" and its associated legend paragraph 23.2.4 [associative.reqmts] p8 omits to impose constraints related to `const` values, contrary to unordered containers as specified by Table 102 and its associated legend paragraph p11.

Reading these requirements strictly, a feasible associative container could declare several conceptual observer members — for example `key_comp()`, `value_comp()`, or `count()` — as non-`const` functions. In Table 102, "possibly `const`" values are exposed by different symbols, so the situation for unordered containers is clear that these functions may be invoked by `const` container objects.

For the above mentioned member functions this problem is only minor, because the synopses of the actual Standard associative containers do declare these members as `const` functions, but nonetheless a wording fix is recommended to clean up the specification asymmetry between associative containers and unordered containers.

The consequences of the ignorance of `const` becomes much worse when we consider a code example such as the following one from a recent libstdc++ bug report:

```
#include <set>

struct compare
{
  bool operator()(int a, int b) // Notice the non-const member declaration!
  {
    return a < b;
  }
```

```
};

int main() {
  const std::set<int, compare> s;
  s.find(0);
}
```

The current wording in 23.2.4 [associative.reqmts] can be read to require this code to be well-formed, because there is no requirement that an object `comp` of the ordering relation of type `Compare` might be a `const` value when the function call expression `comp(k1, k2)` is evaluated.

Current implementations differ: While Clangs libc++ and GCCs libstdc++ reject the above example, the Dinkumware library associated with Visual Studio 2015 accepts it.

I believe the current wording unintentionally misses the constraint that even `const` comparison function objects of associative containers need to support the predicate call expression. This becomes more obvious when considering the member `value_compare` of `std::map` which provides (only) a `const operator()` overload which invokes the call expression of data member `comp`.

[2016-02-20, Daniel comments and extends suggested wording]

It has been pointed out to me, that the suggested wording is a potentially breaking change and should therefore be mentioned in Annex C.

First, let me emphasize that this potentially breaking change is *solely* caused by the wording change in 23.2.4 [associative.reqmts] p8:

> [...] and `c` denotes a possibly `const` value of type `X::key_compare`; [...]

So, even if that proposal would be rejected, the rest of the suggested changes could (and should) be considered for further evaluation, because the remaining parts do just repair an obvious mismatch between the concrete associative containers (`std::set`, `std::map`, ...) and the requirement tables.

Second, I believe that the existing wording was never really clear in regard to *require* a Standard Library to accept comparison functors with non-`const operator()`. If the committee really intends to require a Library to support comparison functors with non-`const operator()`, this should be clarified by at least an additional note to e.g. 23.2.4 [associative.reqmts] p8.

[2016-03, Jacksonville]

Move to Ready with Daniel's updated wording

Proposed resolution:

This wording is relative to N4567.

1. Change 23.2.4 [associative.reqmts] p8 as indicated:

> -8- In Table 101, `X` denotes an associative container class, `a` denotes a value of type `X`, `b` denotes a possibly `const` value of type `X`, `u` denotes the name of a variable being declared, `a_uniq` denotes a value of type `X` when `X` supports unique keys, `a_eq` denotes a value of type `X` when `X` supports multiple keys, `a_tran` denotes a possibly `const` value of type `X` when the *qualified-id* `X::key_compare::is_transparent` is valid and denotes a type (14.8.2), `i` and `j` satisfy input iterator requirements and refer to elements implicitly convertible to `value_type`, `[i, j)` denotes a valid range, `p` denotes a valid const iterator to `a`, `q` denotes a valid dereferenceable const iterator to `a`, `r` denotes a valid dereferenceable iterator to `a`, `[q1, q2)` denotes a valid range of const iterators in `a`, `il` designates an object of type `initializer_list<value_type>`, `t` denotes a value of type `X::value_type`, `k` denotes a value of type `X::key_type` and `c` denotes a possibly `const` value of type `X::key_compare`; `kl` is a value such that `a` is partitioned (25.4) with respect to `c(r, kl)`, with `r` the key value of `e` and `e` in `a`; `ku` is a value such that `a` is partitioned with respect to `!c(ku, r)`; `ke` is a value such that `a` is partitioned with respect to `c(r, ke)` and `!c(ke, r)`, with `c(r, ke)` implying `!c(ke, r)`. `A` denotes the storage allocator used by `X`, if any, or `std::allocator<X::value_type>` otherwise, and `m` denotes an allocator of a type convertible to `A`.

2. Change 23.2.4 [associative.reqmts], Table 101 — "Associative container requirements" as indicated:

[*Editorial note*: This issue doesn't touch the note column entries for the expressions related to `key_comp()` and `value_comp()` (except for the symbolic correction), since these are already handled by LWG issues 2227 and 2215 — *end editorial note*]

Table 101 — Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-	Complexity

		condition	
		…	
~~ab~~.key_comp()	X::key_compare	returns the comparison object out of which ~~ab~~ was constructed.	constant
~~ab~~.value_comp()	X::value_compare	returns an object of value_compare constructed out of the comparison object	constant
		…	
~~ab~~.find(k)	iterator; const_iterator for constant ~~ab~~.	returns an iterator pointing to an element with the key equivalent to k, or ~~ab~~.end() if such an element is not found	logarithmic
		…	
~~ab~~.count(k)	size_type	returns the number of elements with key equivalent to k	log(~~ab~~.size()) + ~~ab~~.count(k)
		…	
~~ab~~.lower_bound(k)	iterator; const_iterator for constant ~~ab~~.	returns an iterator pointing to the first element with key not less than k, or ~~ab~~.end() if such an element is not found.	logarithmic
		…	
~~ab~~.upper_bound(k)	iterator; const_iterator for constant ~~ab~~.	returns an iterator pointing to the first element with key greater than k, or ~~ab~~.end() if such an element is not found.	logarithmic
		…	
~~ab~~.equal_range(k)	pair<iterator, iterator>; pair<const_iterator, const_iterator> for constant ~~ab~~.	equivalent to make_-pair(~~ab~~.lower_bound(k), ~~ab~~.upper_bound(k)).	logarithmic
		…	

3. Add a new entry to Annex C, C.4 [diff.cpp14], as indicated:

C.4.4 Clause 23: containers library [diff.cpp14.containers]

23.2.4

Change: Requirements change:

Rationale: Increase portability, clarification of associative container requirements.

Effect on original feature: Valid 2014 code that attempts to use associative containers having a comparison object with non-const function call operator may fail to compile:

```
#include <set>

struct compare
{
  bool operator()(int a, int b)
  {
    return a < b;
  }
};

int main() {
  const std::set<int, compare> s;
  s.find(0);
}
```

2549. Tuple *EXPLICIT* constructor templates that take `tuple` parameters end up taking references to temporaries and will create dangling references

Section: 20.4.2.1 [tuple.cnstr] **Status:** Ready **Submitter:** Ville Voutilainen **Opened:** 2015-10-11 **Last modified:** 2016-03-08

Priority: 2

View other active issues in [tuple.cnstr].

View all other issues in [tuple.cnstr].

View all issues with Ready status.

Discussion:

Consider this example:

```
#include <utility>
#include <tuple>

struct X {
  int state; // this has to be here to not allow tuple
             // to inherit from an empty X.
  X() {
  }

  X(X const&) {
  }

  X(X&&) {
  }

  ~X() {
  }
};

int main()
{
  X v;
  std::tuple<X> t1{v};
  std::tuple<std::tuple<X>&&> t2{std::move(t1)}; // #1
  std::tuple<std::tuple<X>> t3{std::move(t2)}; // #2
}
```

The line marked with `#1` will use the constructor `template <class... UTypes> EXPLICIT constexpr tuple(tuple<UTypes...>&& u);` and will construct a temporary and bind an rvalue reference to it. The line marked with `#2` will move from a dangling reference.

In order to solve the problem, the constructor templates taking tuples as parameters need additional SFINAE conditions that SFINAE those constructor templates away when `Types...` is constructible or convertible from the incoming tuple type and `sizeof...(Types)` equals one. libstdc++ already has this fix applied.

There is an additional check that needs to be done, in order to avoid infinite meta-recursion during overload resolution, for a case where the element type is itself constructible from the target tuple. An example illustrating that problem is as follows:

```
#include <tuple>

struct A
{
  template <typename T>
  A(T)
  {
  }

  A(const A&) = default;
  A(A&&) = default;
  A& operator=(const A&) = default;
  A& operator=(A&&) = default;
  ~A() = default;
};

int main()
{
  auto x = A{7};
  std::make_tuple(x);
}
```

I provide two proposed resolutions, one that merely has a note encouraging trait-based implementations to avoid infinite meta-recursion, and a second one that avoids it normatively (implementations can still do things differently under the as-if rule, so we

are not necessarily overspecifying how to do it.)

[2016-02-17, Ville comments]

It was pointed out at gcc bug 69853 that the fix for LWG 2549 is a breaking change. That is, it breaks code that expects constructors inherited from `tuple` to provide an implicit base-to-derived conversion. I think that's just additional motivation to apply the fix; that conversion is very much undesirable. The example I wrote into the bug report is just one example of a very subtle temporary being created.

Previous resolution from Ville [SUPERSEDED]:

A. **Alternative 1:**

1. In 20.4.2.1 [tuple.cnstr]/17, edit as follows:

```
template <class... UTypes> EXPLICIT constexpr tuple(const tuple<UTypes...>& u);
```

[...]

-17- *Remarks*: This constructor shall not participate in overload resolution unless

- `is_constructible<T`$_i$`, const Ui&>::value` is true for all i~~,~~, and
- `sizeof...(Types) != 1`, or `is_convertible<const tuple<UTypes...>&, Types...>::value` is false and `is_constructible<Types..., const tuple<UTypes...>&>::value` is false.

[*Note*: to avoid infinite template recursion in a trait-based implementation for the case where `UTypes...` is a single type that has a constructor template that accepts `tuple<Types...>`, implementations need to additionally check that `remove_cv_t<remove_reference_t<const tuple<UTypes...>&>>` and `tuple<Types...>` are not the same type. — *end note*]

The constructor is explicit if and only if `is_convertible<const U`$_i$`&, T`$_i$`>::value` is false for at least one i.

2. In 20.4.2.1 [tuple.cnstr]/20, edit as follows:

```
template <class... UTypes> EXPLICIT constexpr tuple(tuple<UTypes...>&& u);
```

[...]

-20- *Remarks*: This constructor shall not participate in overload resolution unless

- `is_constructible<T`$_i$`, U`$_i$`&&>::value` is true for all i~~,~~, and
- `sizeof...(Types) != 1`, or `is_convertible<tuple<UTypes...>&&, Types...>::value` is false and `is_constructible<Types..., tuple<UTypes...>&&>::value` is false.

[*Note*: to avoid infinite template recursion in a trait-based implementation for the case where `UTypes...` is a single type that has a constructor template that accepts `tuple<Types...>`, implementations need to additionally check that `remove_cv_t<remove_reference_t<tuple<UTypes...>&&>>` and `tuple<Types...>` are not the same type. — *end note*]

The constructor is explicit if and only if `is_convertible<U`$_i$`&&, T`$_i$`>::value` is false for at least one i.

B. **Alternative 2 (do the sameness-check normatively):**

1. In 20.4.2.1 [tuple.cnstr]/17, edit as follows:

```
template <class... UTypes> EXPLICIT constexpr tuple(const tuple<UTypes...>& u);
```

[...]

-17- *Remarks*: This constructor shall not participate in overload resolution unless

- `is_constructible<T`$_i$`, const Ui&>::value` is true for all i~~,~~, and

- `sizeof...(Types) != 1` or `is_convertible<const tuple<UTypes...>&, Types...>::value` is false and `is_constructible<Types..., const tuple<UTypes...>&>::value` is false and `is_same<tuple<Types...>, remove_cv_t<remove_reference_t<const tuple<UTypes...>&>>>::value` is false.

The constructor is explicit if and only if `is_convertible<const` U_i`&, ` T_i`>::value` is false for at least one i.

2. In 20.4.2.1 [tuple.cnstr]/20, edit as follows:

```
template <class... UTypes> EXPLICIT constexpr tuple(tuple<UTypes...>&& u);
```

[...]

-20- *Remarks*: This constructor shall not participate in overload resolution unless

- `is_constructible<`T_i`, `U_i`&&>::value` is true for all i, and
- `sizeof...(Types) != 1` or `is_convertible<tuple<UTypes...>&&, Types...>::value` is false and `is_constructible<Types..., tuple<UTypes...>&&>::value` is false and `is_same<tuple<Types...>, remove_cv_t<remove_reference_t<tuple<UTypes...>&&>>>::value` is false.

The constructor is explicit if and only if `is_convertible<`U_i`&&, `T_i`>::value` is false for at least one i.

[2016-03, Jacksonville]

STL provides a simplification of Ville's alternative #2 (with no semantic changes), and it's shipping in VS 2015 Update 2.

Proposed resolution:

This wording is relative to N4567.

A. This approach is orthogonal to the Proposed Resolution for LWG 2312:

1. Edit 20.4.2.1 [tuple.cnstr] as indicated:

```
template <class... UTypes> EXPLICIT constexpr tuple(const tuple<UTypes...>& u);
```

[...]

-17- *Remarks*: This constructor shall not participate in overload resolution unless

- `is_constructible<`T_i`, const `U_i`&>::value` is `true` for all i, and
- `sizeof...(Types) != 1` or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `!is_convertible_v<const tuple<U>&, T> && !is_constructible_v<T, const tuple<U>&> && !is_same_v<T, U>` is `true`.

The constructor is explicit if and only if `is_convertible<const` U_i`&, ` T_i`>::value` is `false` for at least one i.

```
template <class... UTypes> EXPLICIT constexpr tuple(tuple<UTypes...>&& u);
```

[...]

-20- *Remarks*: This constructor shall not participate in overload resolution unless

- `is_constructible<`T_i`, `U_i`&&>::value` is `true` for all i, and
- `sizeof...(Types) != 1` or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `!is_convertible_v<tuple<U>, T> && !is_constructible_v<T, tuple<U>> && !is_same_v<T, U>` is `true`.

The constructor is explicit if and only if `is_convertible<`U_i`&&, `T_i`>::value` is `false` for at least one i.

B. This approach is presented as a merge with the parts of the Proposed Resolution for LWG 2312 with overlapping modifications in the same paragraph, to provide editorial guidance if 2312 would be accepted.

1. Edit 20.4.2.1 [tuple.cnstr] as indicated:

```
template <class... UTypes> EXPLICIT constexpr tuple(const tuple<UTypes...>& u);
```

[...]

-17- *Remarks*: This constructor shall not participate in overload resolution unless

- `sizeof...(Types) == sizeof...(UTypes)`, and
- `is_constructible<`T_i`, const `U_i`&>::value` is `true` for all i, and
- `sizeof...(Types) != 1`, or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `!is_convertible_v<const tuple<U>&, T> && !is_constructible_v<T, const tuple<U>&> && !is_same_v<T, U>` is `true`.

The constructor is explicit if and only if `is_convertible<const `U_i`&, `T_i`>::value` is `false` for at least one i.

```
template <class... UTypes> EXPLICIT constexpr tuple(tuple<UTypes...>&& u);
```

[...]

-20- *Remarks*: This constructor shall not participate in overload resolution unless

- `sizeof...(Types) == sizeof...(UTypes)`, and
- `is_constructible<`T_i`, `U_i`&&>::value` is `true` for all i, and
- `sizeof...(Types) != 1`, or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `!is_convertible_v<tuple<U>, T> && !is_constructible_v<T, tuple<U>> && !is_same_v<T, U>` is `true`.

The constructor is explicit if and only if `is_convertible<`U_i`&&, `T_i`>::value` is `false` for at least one i.

2550. Wording of unordered container's `clear()` method complexity

Section: 23.2.5 [unord.req] **Status:** Ready **Submitter:** Yegor Derevenets **Opened:** 2015-10-11 **Last modified:** 2016-03-06

Priority: 2

View other active issues in [unord.req].

View all other issues in [unord.req].

View all issues with Ready status.

Discussion:

I believe, the wording of the complexity specification for the standard unordered containers' `clear()` method should be changed from "Linear" to "Linear in `a.size()`". As of N4527, the change should be done in the Complexity column of row "`a.clear()`..." in "Table 102 — Unordered associative container requirements" in section Unordered associative containers 23.2.5 [unord.req].

From the current formulation it is not very clear, whether the complexity is linear in the number of buckets, in the number of elements, or both. cppreference is also not very specific: it mentions the size of the container without being explicit about what exactly the size is.

The issue is inspired by a performance bug in libstdc++. The issue is related to LWG 1175.

[2016-03 Jacksonville]

GR: erase(begin. end) has to touch every element. clear() has the option of working with buckets instead. will be faster in some cases, slower in some. clear() has to be at least linear in size as it has to run destructors.
MC: wording needs to say what it's linear in, either elements or buckets.
HH: my vote is the proposed resolution is correct.
Move to Ready.

Proposed resolution:

This wording is relative to N4527.

1. Change 23.2.5 [unord.req], Table 102 as indicated:

Table 102 — Unordered associative container requirements (in addition to container)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
		...	
`a.clear()`	`void`	Erases all elements in the container. Post: `a.empty()` returns `true`	Linear in `a.size()`.
		...	

2551. [fund.ts.v2] "Exception safety" cleanup in library fundamentals required

Section: 8.2.1.1 [fund.ts.v2::memory.smartptr.shared.const] **Status:** Ready **Submitter:** Daniel Krügler **Opened:** 2015-10-24 **Last modified:** 2016-03-06

Priority: 0

View all issues with Ready status.

Discussion:

Addresses: fund.ts.v2

Similar to LWG 2495, the current library fundamentals working paper refers to several "Exception safety" elements without providing a definition for this element type.

Proposed resolution:

This wording is relative to N4529.

1. Change 8.2.1.1 [memory.smartptr.shared.const] as indicated:

```
template<class Y> explicit shared_ptr(Y* p);
```

[...]

-3- *Effects*: When `T` is not an array type, constructs a `shared_ptr` object that *owns* the pointer `p`. Otherwise, constructs a `shared_ptr` that *owns* `p` and a deleter of an unspecified type that calls `delete[] p`. If an exception is thrown, `delete p` is called when `T` is not an array type, `delete[] p` otherwise.

[...]

~~-6- *Exception safety*: If an exception is thrown, `delete p` is called when `T` is not an array type, `delete[] p` otherwise.~~

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

[...]

-9- *Effects*: Constructs a `shared_ptr` object that *owns* the object `p` and the deleter `d`. The second and fourth constructors shall use a copy of `a` to allocate memory for internal use. If an exception is thrown, `d(p)` is called.

[...]

~~-12- *Exception safety*: If an exception is thrown, `d(p)` is called.~~

[...]

```
template<class Y> explicit shared_ptr(const weak_ptr<Y>& r);
```

[...]

-28- *Effects*: Constructs a `shared_ptr` object that *shares ownership* with `r` and stores a copy of the pointer stored in `r`. If an exception is thrown, the constructor has no effect.

[...]

~~-31- *Exception safety*: If an exception is thrown, the constructor has no effect.~~

```
template <class Y, class D> shared_ptr(unique_ptr<Y, D>&& r);
```

[...]

-34- *Effects*: Equivalent to `shared_ptr(r.release(), r.get_deleter())` when `D` is not a reference type, otherwise `shared_ptr(r.release(), ref(r.get_deleter()))`. If an exception is thrown, the constructor has no effect.

~~-35- *Exception safety*: If an exception is thrown, the constructor has no effect.~~

2555. [fund.ts.v2] No handling for over-aligned types in `optional`

Section: 5.3 [fund.ts.v2::optional.object] **Status:** Ready **Submitter:** Marshall Clow **Opened:** 2015-11-03 **Last modified:** 2016-03-06

Priority: 0

View other active issues in [fund.ts.v2::optional.object].

View all other issues in [fund.ts.v2::optional.object].

View all issues with Ready status.

Discussion:

Addresses: fund.ts.v2

5.3 [optional.object] does not specify whether over-aligned types are supported. In other places where we specify allocation of user-supplied types, we state that "It is implementation-defined whether over-aligned types are supported (3.11)." (Examples: 5.3.4 [expr.new]/p1, 20.9.9.1 [allocator.members]/p5, 20.9.11 [temporary.buffer]/p1). We should presumably do the same thing here.

Proposed resolution:

This wording is relative to N4562.

1. Edit 5.3 [optional.object]/p1 as follows::

   [...] The contained value shall be allocated in a region of the `optional<T>` storage suitably aligned for the type `T`. It is implementation-defined whether over-aligned types are supported (C++14 §3.11). When an object of type `optional<T>` is contextually converted to `bool`, the conversion returns `true` if the object contains a value; otherwise the conversion returns `false`.

2573. [fund.ts.v2] `std::hash<std::experimental::shared_ptr<T>>` does not work for arrays

Section: 8.2.1 [fund.ts.v2::memory.smartptr.shared] **Status:** Ready **Submitter:** Tim Song **Opened:** 2015-12-13 **Last modified:** 2016-03-06

Priority: 0

View all issues with Ready status.

Discussion:

Addresses: fund.ts.v2

The library fundamentals TS does not provide a separate specification for `std::hash<std::experimental::shared_ptr<T>>`, deferring to the C++14 specification in §20.8.2.7/3:

```
template <class T> struct hash<shared_ptr<T> >;
```

-3- The template specialization shall meet the requirements of class template `hash` (20.9.13). For an object `p` of type `shared_ptr<T>`, `hash<shared_ptr<T> >()(p)` shall evaluate to the same value as `hash<T*>()(p.get())`.

That specification doesn't work if `T` is an array type (`U[N]` or `U[]`), as in this case `get()` returns `U*`, which cannot be hashed by `std::hash<T*>`.

Proposed resolution:

This wording is relative to N4562.

1. Insert a new subclause after 8.2.1.3 [memory.smartptr.shared.cast]:

> [*Note for the editor*: The synopses in [header.memory.synop] and [memory.smartptr.shared] should be updated to refer to the new subclause rather than C++14 §20.8.2.7]

?.?.?.? `shared_ptr` hash support [memory.smartptr.shared.hash]

```
template <class T> struct hash<experimental::shared_ptr<T>>;
```

-1- The template specialization shall meet the requirements of class template `hash` (C++14 §20.9.12). For an object `p` of type `experimental::shared_ptr<T>`, `hash<experimental::shared_ptr<T>>()(p)` shall evaluate to the same value as `hash<typename experimental::shared_ptr<T>::element_type*>()(p.get())`.

2596. `vector::data()` should use `addressof`

Section: 23.3.11.4 [vector.data] **Status:** Tentatively Ready **Submitter:** Marshall Clow **Opened:** 2016-02-29 **Last modified:** 2016-04-15

Priority: 0

View all other issues in [vector.data].

View all issues with Tentatively Ready status.

Discussion:

In 23.3.11.4 [vector.data], we have:

> *Returns*: A pointer such that `[data(),data() + size())` is a valid range. For a non-empty vector, `data() == &front()`.

This should be:

> *Returns*: A pointer such that `[data(),data() + size())` is a valid range. For a non-empty vector, `data() == addressof(front())`.

Proposed resolution:

This wording is relative to N4582.

1. Change 23.3.11.4 [vector.data] p1 as indicated:

```
T* data() noexcept;
const T* data() const noexcept;
```

-1- *Returns*: A pointer such that `[data(), data() + size())` is a valid range. For a non-empty vector, `data() == addressof(&front())`.

2667. `path::root_directory()` description is confusing

Section: 27.10.8.4.9 [path.decompose] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2014-07-03 **Last modified:** 2016-04-14

Priority: 0

View all issues with Ready status.

Discussion:

27.10.8.4.9 [path.decompose] p5 says:

> If *root-directory* is composed of *slash name*, *slash* is excluded from the returned string.

but the grammar says that *root-directory* is just slash so that makes no sense.

[Apr 2016 Issue updated to address the C++ Working Paper. Previously addressed File System TS]

Proposed resolution:

Change 27.10.8.4.9 [path.decompose] as indicated:

> `path root_directory() const;`
>
> *Returns: root-directory*, if `pathname` includes *root-directory*, otherwise `path()`.
>
> ~~If *root-directory* is composed of *slash name*, *slash* is excluded from the returned string.~~

2669. `recursive_directory_iterator` effects refers to non-existent functions

Section: 27.10.14.1 [rec.dir.itr.members] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2014-07-10 **Last modified:** 2016-04-14

Priority: 0

View other active issues in [rec.dir.itr.members].

View all other issues in [rec.dir.itr.members].

View all issues with Ready status.

Discussion:

`operator++/increment`, second bullet item, ¶39 says "Otherwise if `recursion_pending() && is_directory(this->status()) && (!is_symlink(this->symlink_status())...`" but `recursive_directory_iterator` does not have `status` or `symlink_status` members.

[Apr 2016 Issue updated to address the C++ Working Paper. Previously addressed File System TS]

Proposed resolution:

Change 27.10.14.1 [rec.dir.itr.members] ¶39 as indicated:

> Otherwise if `recursion_pending() && is_directory((*this)->status()) && (!is_symlink((*this)->symlink_status()) ...`"

2670. `system_complete` refers to undefined variable 'base'

Section: 27.10.15.36 [fs.op.system_complete] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2014-07-20 **Last modified:** 2016-04-14

Priority: 0

View all issues with Ready status.

Discussion:

The example says "...or `p` and `base` have the same `root_name()`.", but `base` is not defined. I believe it refers to the value returned by `current_path()`.

[Apr 2016 Issue updated to address the C++ Working Paper. Previously addressed File System TS]

Proposed resolution:

Change 27.10.15.36 [fs.op.system_complete] as indicated:

> For Windows based operating systems, `system_complete(p)` has the same semantics as `absolute(p, current_path())` if `p.is_absolute() || !p.has_root_name()` or `p` and ~~`base`~~ `current_path()` have the same `root_name()`. Otherwise it acts like `absolute(p, cwd)` is the current directory for the `p.root_name()` drive. This will be the current directory for that drive the last time it was set, and thus may be residue left over from a prior program run by the command processor.

Although these semantics are useful, they may be surprising.

2671. Errors in Copy

Section: 27.10.15.3 [fs.op.copy] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2014-07-28 **Last modified:** 2016-04-14

Priority: 0

View other active issues in [fs.op.copy].

View all other issues in [fs.op.copy].

View all issues with Ready status.

Discussion:

27.10.15.3 [fs.op.copy] paragraph 16 says:

> Otherwise if `!exists(t) & (options & copy_options::copy_symlinks) != copy_options::none`, then `copy_symlink(from, to, options)`.

But there is no overload of `copy_symlink` that takes a `copy_options`; it should be `copy_symlink(from, to)`.

27.10.15.3 [fs.op.copy] Paragraph 26 says:

> as if by `for (directory_entry& x : directory_iterator(from))`

but the result of dereferencing a `directory_iterator` is const; it should be:

> as if by `for (const directory_entry& x : directory_iterator(from))`

[Apr 2016 Issue updated to address the C++ Working Paper. Previously addressed File System TS]

Proposed resolution:

Change 27.10.15.3 [fs.op.copy] paragraph 16 as indicated:

> Otherwise if `!exists(t) & (options & copy_options::copy_symlinks) != copy_options::none` , then `copy_symlink(from, to~~, options~~)`.

Change 27.10.15.3 [fs.op.copy] paragraph 26 as indicated:

> as if by `for (const directory_entry& x : directory_iterator(from))`

2673. `status()` effects cannot be implemented as specified

Section: 27.10.15.33 [fs.op.status] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2015-09-15 **Last modified:** 2016-04-14

Priority: 0

View all issues with Ready status.

Discussion:

27.10.15.33 [fs.op.status] paragraph 2 says:

> *Effects:* As if:

```
error_code ec;
file_status result = status(p, ec);
if (result == file_type::none)
...
```

This won't compile, there is no comparison operator for `file_status` and `file_type`, and the conversion from `file_type` to `file_status` is explicit.

[Apr 2016 Issue updated to address the C++ Working Paper. Previously addressed File System TS]

Proposed resolution:

Change 27.10.15.33 [fs.op.status] paragraph 2:

> *Effects:* As if:
>
> ```
> error_code ec;
> file_status result = status(p, ec);
> if (result.type() == file_type::none)
> ...
> ```

2674. Bidirectional iterator requirement on `path::iterator` is very expensive

Section: 27.10.8.5 [path.itr] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2015-09-15 **Last modified:** 2016-05-17

Priority: 2

View all issues with Tentatively Ready status.

Discussion:

27.10.8.5 [path.itr] requires `path::iterator` to be a BidirectionalIterator, which also implies the ForwardIterator requirement in [forward.iterators] p6 for the following assertion to pass:

```
path p("/");
auto it1 = p.begin();
auto it2 = p.begin();
assert( &*it1 == &*it2 );
```

This prevents iterators containing a `path`, or constructing one on the fly when dereferenced, the object they point to must exist outside the iterators and potentially outlive them. The only practical way to meet the requirement is for `p` to hold a container of child `path` objects so the iterators can refer to those children. This makes a `path` object much larger than would naïvely be expected.

The Boost and MSVC implementations of Filesystem fail to meet this requirement. The GCC implementation meets it, but it makes `sizeof(path) == 64` (for 64-bit) or `sizeof(path) == 40` for 32-bit, and makes many path operations more expensive.

[21 Nov 2015 Beman comments:]

The ForwardIterator requirement in [forward.iterators] "If `a` and `b` are both dereferenceable, then `a == b` if and only if `*a` and `*b` are bound to the same object." will be removed by N4560, Working Draft, C++ Extensions for Ranges. I see no point in requiring something for the File System TS that is expensive, has never to my knowledge been requested by users, and is going to go away soon anyhow. The wording I propose below removes the requirement.

[Apr 2016 Issue updated to address the C++ Working Paper. Previously addressed File System TS]

Proposed resolution:

This wording is relative to N4582.

1. Change 27.10.8.5 [path.itr] paragraph 2:

> A `path::iterator` is a constant iterator satisfying all the requirements of a bidirectional iterator (C++14 §24.1.4 Bidirectional iterators) except that there is no requirement that two equal iterators be bound to the same object. Its `value_type` is `path`.

2683. `filesystem::copy()` says "no effects"

Section: 27.10.15.3 [fs.op.copy] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2016-04-19 **Last modified:** 2016-05-22

Priority: 0

View other active issues in [fs.op.copy].

View all other issues in [fs.op.copy].

View all issues with Tentatively Ready status.

Discussion:

In 27.10.15.3 [fs.op.copy]/8 the final bullet says "Otherwise, no effects" which implies there is no call to `ec.clear()` if nothing happens, nor any error condition, is that right?

Proposed resolution:

Change 27.10.15.3 [fs.op.copy]/8 as indicated:

> ~~Otherwise no effects.~~ For the signature with argument `ec`, `ec.clear()`.

2684. priority_queue lacking comparator typedef

Section: 23.6.5 [priority.queue] **Status:** Tentatively Ready **Submitter:** Robert Haberlach **Opened:** 2016-05-02 **Last modified:** 2016-05-22

Priority: 0

View all other issues in [priority.queue].

View all issues with Tentatively Ready status.

Discussion:

The containers that take a comparison functor (`set`, `multiset`, `map`, and `multimap`) have a typedef for the comparison functor. `priority_queue` does not.

Proposed resolution:

Augment [priority.queue] as indicated:

```
typedef Container container_type;
typedef Compare value_compare;
```

2685. `shared_ptr` deleters must not not throw on move construction

Section: 20.10.2.2.1 [util.smartptr.shared.const] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2016-05-03 **Last modified:** 2016-05-22

Priority: 0

View all other issues in [util.smartptr.shared.const].

View all issues with Tentatively Ready status.

Discussion:

In 20.10.2.2.1 [util.smartptr.shared.const] p8 the `shared_ptr` constructors taking a deleter say:

> The copy constructor and destructor of `D` shall not throw exceptions.

It's been pointed out that this doesn't forbid throwing moves, which makes it difficult to avoid a leak here:

```
struct D {
  D() = default;
  D(const D&) noexcept = default;
  D(D&&) { throw 1; }
  void operator()(int* p) const { delete p; }
};

shared_ptr<int> p{new int, D{}};
```

"The copy constructor" should be changed to reflect that the chosen constructor might not be a copy constructor, and that copies made using any constructor must not throw.

N.B. the same wording is used for the allocator argument, but that's redundant because the `Allocator` requirements already forbid exceptions when copying or moving.

Proposed resolution:

[Drafting note: the relevant expressions we're concerned about are enumerated in the `CopyConstructible` and `MoveConstructible` requirements, so I see no need to repeat them by saying something clunky like "Initialization of an object of type `D` from an expression of type (possibly const) `D` shall not throw exceptions", we can just refer to them. An alternative would be to define `NothrowCopyConstructible`, which includes `CopyConstructible` but requires that construction and destruction do not throw.]

Change 20.10.2.2.1 [util.smartptr.shared.const] p8:

> `D` shall be `CopyConstructible` and such construction shall not throw exceptions. The ~~copy constructor and~~ destructor of `D` shall not throw exceptions.

2688. `clamp` misses preconditions and has extraneous condition on result

Section: 25.5.8 [alg.clamp] **Status:** Tentatively Ready **Submitter:** Martin Moene **Opened:** 2016-03-23 **Last modified:** 2016-05-22

Priority: 0

View all issues with Tentatively Ready status.

Discussion:

In Jacksonville (2016), P0025R0 was voted in instead of the intended P0025R1. This report contains the necessary mending along with two other improvements.

This report:

- adds the precondition that misses from P0025R0 but is in P0025R1,
- corrects the returns: specification that contains an extraneous condition,
- replaces the now superfluous remark with a note on usage of clamp with `float` or `double`.

Thanks to Carlo Assink and David Gaarenstroom for making us aware of the extraneous condition in the returns: specification and for suggesting the fix and to Jeffrey Yasskin for suggesting to add a note like p3.

[2016-05 Issues Telecom]

Reworded p3 slightly.

Proposed resolution:

This wording is relative to N4582.

Previous resolution [SUPERSEDED]:

1. Edit 25.5.8 [alg.clamp] as indicated:

```
template<class T>
  constexpr const T& clamp(const T& v, const T& lo, const T& hi);
template<class T, class Compare>
  constexpr const T& clamp(const T& v, const T& lo, const T& hi, Compare comp);
```

-1- *Requires*: The value of `lo` shall be no greater than `hi`. For the first form, type `T` shall be `LessThanComparable` (Table 18).

-2- *Returns*: `lo` if `v` is less than `lo`, `hi` if `hi` is less than `v`, otherwise `v`~~The larger value of `v` and `lo` if `v` is smaller than `hi`, otherwise the smaller value of `v` and `hi`~~.

-3- *Note*~~*Remarks*~~: If NaN is avoided, `T` can be `float` or `double`~~Returns the first argument when it is equivalent to one of the boundary arguments~~.

-4- *Complexity*: At most two comparisons.

1. Edit 25.5.8 [alg.clamp] as indicated:

```
template<class T>
  constexpr const T& clamp(const T& v, const T& lo, const T& hi);
template<class T, class Compare>
  constexpr const T& clamp(const T& v, const T& lo, const T& hi, Compare comp);
```

-1- *Requires*: The value of `lo` shall be no greater than `hi`. For the first form, type `T` shall be `LessThanComparable` (Table 18).

-2- *Returns*: `lo` if `v` is less than `lo`, `hi` if `hi` is less than `v`, otherwise `v`~~The larger value of `v` and `lo` if `v` is smaller than `hi`, otherwise the smaller value of `v` and `hi`~~.

-3- *Note*~~*Remarks*~~: If NaN is avoided, `T` can be a floating point type ~~Returns the first argument when it is equivalent to one of the boundary arguments~~.

-4- *Complexity*: At most two comparisons.

2689. Parallel versions of `std::copy` and `std::move` shouldn't be in order

Section: 25.4.1 [alg.copy], 25.4.2 [alg.move] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2016-03-23 **Last modified:** 2016-05-22

Priority: 0

View other active issues in [alg.copy].

View all other issues in [alg.copy].

View all issues with Tentatively Ready status.

Discussion:

25.2.5 [algorithms.parallel.overloads]/2 says that "Unless otherwise specified, the semantics of `ExecutionPolicy` algorithm overloads are identical to their overloads without."

There's no "otherwise specified" for the `ExecutionPolicy` overloads for `std::copy` and `std::move`, so the requirement that they "start[] from first and proceed[] to last" in the original algorithm's description would seem to apply, which defeats the whole point of adding a parallel overload.

[2016-05 Issues Telecom]

Marshall noted that all three versions of copy have subtly different wording, and suggested that they should not.

Proposed resolution:

This wording is relative to N4582.

1. Insert the following paragraphs after 25.4.1 [alg.copy]/4:

```
template<class ExecutionPolicy, class InputIterator, class OutputIterator>
  OutputIterator copy(ExecutionPolicy&& policy, InputIterator first, InputIterator last,
                      OutputIterator result);
```

-?- *Requires*: The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.

-?- *Effects*: Copies elements in the range `[first, last)` into the range `[result, result + (last - first))`. For each non-negative integer `n < (last - first)`, performs `*(result + n) = *(first + n)`.

-?- *Returns*: `result + (last - first)`.

-?- *Complexity*: Exactly `last - first` assignments.

2. Insert the following paragraphs after 25.4.2 [alg.move]/4:

```
template<class ExecutionPolicy, class InputIterator, class OutputIterator>
  OutputIterator move(ExecutionPolicy&& policy, InputIterator first, InputIterator last,
                      OutputIterator result);
```

-?- *Requires*: The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.

-?- *Effects*: Moves elements in the range `[first, last)` into the range `[result, result + (last - first))`. For each non-negative integer `n < (last - first)`, performs `*(result + n) = std::move(*(first + n))`.

-?- *Returns*: `result + (last - first)`.

-?- *Complexity*: Exactly `last - first` assignments.

2698. Effect of `assign()` on iterators/pointers/references

Section: 23.2.3 [sequence.reqmts] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2016-04-25 **Last modified:** 2016-05-22

Priority: 0

View other active issues in [sequence.reqmts].

View all other issues in [sequence.reqmts].

View all issues with Tentatively Ready status.

Discussion:

The sequence container requirements table says nothing about the effect of `assign()` on iterators, pointers or references into the container. Before LWG 2209 (and LWG 320 for `std::list`), `assign()` was specified as "erase everything then `insert`", which implies wholesale invalidation from the "erase everything" part. With that gone, the blanket "no invalidation" wording in 23.2.1 [container.requirements.general]/12 would seem to apply, which makes absolutely no sense.

The proposed wording below simply spells out the invalidation rule implied by the previous "erase everything" wording.

[2016-05 Issues Telecom]

This is related to 2256

Proposed resolution:

This wording is relative to N4582.

1. In 23.2.3 [sequence.reqmts], edit Table 107 (Sequence container requirements) as indicated:

Table 107 — Sequence container requirements (in addition to container)

Expression	Return type	Assertion/note pre-/post-condition
[…]		
`a.assign(i, j)`	`void`	*Requires*: `T` shall be `EmplaceConstructible` into `X` from `*i` and assignable from `*i`. For `vector`, if the iterator does not meet the forward iterator requirements (24.2.5), `T` shall also be `MoveInsertable` into `X`. Each iterator in the range `[i, j)` shall be dereferenced exactly once. pre: `i`, `j` are not iterators into `a`. Replaces elements in `a` with a copy of `[i, j)`. Invalidates all references, pointers and iterators referring to the elements of `a`. For `vector` and `deque`, also invalidates the past-the-end iterator.
[…]		
`a.assign(n, t)`	`void`	*Requires*: `T` shall be `CopyInsertable` into `X` and `CopyAssignable`. pre: `t` is not a reference into `a`. Replaces elements in `a` with `n` copies of `t`. Invalidates all references, pointers and iterators referring to the elements of `a`. For `vector` and `deque`, also invalidates the past-the-end iterator.

2706. Error reporting for `recursive_directory_iterator::pop()` is under-specified

Section: 27.10.14 [class.rec.dir.itr] **Status:** Tentatively Ready **Submitter:** Eric Fiselier **Opened:** 2016-05-09 **Last modified:** 2016-05-22

Priority: 0

View all issues with Tentatively Ready status.

Discussion:

Unlike `increment`, `pop()` does not specify how it reports errors nor does it provide a `std::error_code` overload. However implementing `pop()` all but requires performing an increment, so it should handle errors in the same way.

Proposed resolution:

This wording is relative to N4582.

1. Change 27.10.14 [class.rec.dir.itr], class `recursive_directory_iterator` synopsis, as indicated:

```
namespace std::filesystem {
  class recursive_directory_iterator {
  public:
    […]
    void pop();
    void pop(error_code& ec);
    void disable_recursion_pending();
    […]
  };
}
```

2. Change 27.10.14.1 [rec.dir.itr.members] as indicated:

```
void pop();
void pop(error_code& ec);
```

-30- *Requires*: `*this != recursive_directory_iterator()`.

-31- *Effects*: If `depth() == 0`, set `*this` to `recursive_directory_iterator()`. Otherwise, cease iteration of the directory currently being iterated over, and continue iteration over the parent directory.

-?- *Throws*: As specified in Error reporting (27.5.6.5 [error.reporting]).

2707. `path` construction and assignment should have "`string_type&&`" overloads

Section: 27.10.8 [class.path] **Status:** Tentatively Ready **Submitter:** Eric Fiselier **Opened:** 2016-05-09 **Last modified:** 2016-05-22

Priority: 0

View other active issues in [class.path].

View all other issues in [class.path].

View all issues with Tentatively Ready status.

Discussion:

Currently construction of a `path` from the native `string_type` always performs a copy, even when that string is passed as a rvalue. This is a large pessimization as paths are commonly constructed from temporary strings.

One pattern I frequently see is:

```
path foo(path const& p) {
  auto s = p.native();
  mutateString(s);
  return s;
}
```

Implementations should be allowed to move from `s` and avoid an unnecessary allocation. I believe `string_type&&` constructor and assignment operator overloads should be added to support this.

Proposed resolution:

This wording is relative to N4582.

1. Change 27.10.8 [class.path], class `path` synopsis, as indicated:

> [*Drafting note*: Making the `string_type&&` constructors and assignment operators `noexcept` would over-constrain implementations which may need to perform construct additional state]

```
namespace std::filesystem {
  class path {
  public:
    […]
    // 27.10.8.4.1, constructors and destructor
```

```
    path() noexcept;
    path(const path& p);
    path(path&& p) noexcept;
    path(string_type&& source);
    template <class Source>
    path(const Source& source);
    […]

    // 27.10.8.4.2, assignments
    path& operator=(const path& p);
    path& operator=(path&& p) noexcept;
    path& operator=(string_type&& source);
    path& assign(string_type&& source);
    template <class Source>
    path& operator=(const Source& source);
    template <class Source>
    path& assign(const Source& source)
    template <class InputIterator>
    path& assign(InputIterator first, InputIterator last);
    […]
  };
}
```

2. Add a new paragraph following 27.10.8.4.1 [path.construct]/3:

`path(string_type&& source);`

-?- *Effects*: Constructs an object of class `path` with `pathname` having the original value of `source`. `source` is left in a valid but unspecified state.

3. Add a new paragraph following 27.10.8.4.2 [path.assign]/4:

`path& operator=(string_type&& source);`
`path& assign(string_type&& source);`

-?- *Effects*: Modifies `pathname` to have the original value of `source`. `source` is left in a valid but unspecified state.

-?- *Returns*:`*this`

2710. "*Effects:* Equivalent to ..." doesn't count "*Synchronization:*" as determined semantics

Section: 17.5.1.4 [structure.specifications] **Status:** Tentatively Ready **Submitter:** Kazutoshi Satoda **Opened:** 2016-05-08 **Last modified:** 2016-05-22

Priority: 0

View other active issues in [structure.specifications].

View all other issues in [structure.specifications].

View all issues with Tentatively Ready status.

Discussion:

From N4582 17.5.1.4 [structure.specifications] p3 and p4

-3- Descriptions of function semantics contain the following elements (as appropriate):

- *Requires*: the preconditions for calling the function
- *Effects*: the actions performed by the function
- *Synchronization*: the synchronization operations (1.10) applicable to the function
- *Postconditions*: the observable results established by the function
- *Returns*: a description of the value(s) returned by the function
- *Throws*: any exceptions thrown by the function, and the conditions that would cause the exception
- *Complexity*: the time and/or space complexity of the function
- *Remarks*: additional semantic constraints on the function
- *Error conditions*: the error conditions for error codes reported by the function.
- *Notes*: non-normative comments about the function

-4- Whenever the *Effects:* element specifies that the semantics of some function `F` are *Equivalent to* some code

sequence, then the various elements are interpreted as follows. If `F`'s semantics specifies a *Requires:* element, then that requirement is logically imposed prior to the *equivalent-to* semantics. Next, the semantics of the code sequence are determined by the *Requires:*, *Effects:*, *Postconditions:*, *Returns:*, *Throws:*, *Complexity:*, *Remarks:*, *Error conditions:*, and *Notes:* specified for the function invocations contained in the code sequence. The value returned from `F` is specified by `F`'s *Returns:* element, or if `F` has no *Returns:* element, a non-`void` return from `F` is specified by the *Returns:* elements in the code sequence. If `F`'s semantics contains a *Throws:*, *Postconditions:*, or *Complexity:* element, then that supersedes any occurrences of that element in the code sequence.

The third sentence of p4 says "the semantics of the code sequence are determined by ..." and lists all elements in p3 except "*Synchronization:*".

I think it was just an oversight because p4 was added by library issue 997, and its proposed resolution was drafted at the time (2009) before "*Synchronization:*" was added into p3 for C++11.

However, I'm not definitely sure that it is really intended and safe to just supply "*Synchronization:*" in the list. (Could a library designer rely on this in writing new specifications, or could someone rely on this in writing user codes, after some years after C++11?)

Proposed resolution:

This wording is relative to N4582.

1. Change 17.5.1.4 [structure.specifications] as indicated:

> -4- Whenever the *Effects:* element specifies that the semantics of some function `F` are *Equivalent to* some code sequence, then the various elements are interpreted as follows. If `F`'s semantics specifies a *Requires:* element, then that requirement is logically imposed prior to the *equivalent-to* semantics. Next, the semantics of the code sequence are determined by the *Requires:*, *Effects:*, *Synchronization:*, *Postconditions:*, *Returns:*, *Throws:*, *Complexity:*, *Remarks:*, *Error conditions:*, and *Notes:* specified for the function invocations contained in the code sequence. The value returned from `F` is specified by `F`'s *Returns:* element, or if `F` has no *Returns:* element, a non-`void` return from `F` is specified by the *Returns:* elements in the code sequence. If `F`'s semantics contains a *Throws:*, *Postconditions:*, or *Complexity:* element, then that supersedes any occurrences of that element in the code sequence.